

CHAPTER 1

INTRODUCTION

1.1 Problem Definition

Managing personal finances in today's digital age has become increasingly complex. People often juggle multiple income sources, countless online transactions, and spending spread across different categories. Traditional methods like notebooks, spreadsheets, or basic tracking apps rarely keep up—they lack automation, strong security, and meaningful insights. Because of this, many individuals struggle to truly understand their financial habits or maintain long-term stability.

Security is another major challenge. Many simple finance tools don't offer proper authentication, leaving sensitive transaction data vulnerable. To solve this, the proposed system introduces secure OTP-based email verification, ensuring that only verified users can access their financial dashboard.

Users also face difficulties with analysis. Without features like real-time budget monitoring, automatic categorization, savings progress visualization, or clear summaries, financial planning often feels confusing and inconsistent. Manually preparing reports and tracking expenses takes time and frequently leads to mistakes. This project addresses these issues with a Smart Financial Planning and Transaction Categorization System.

analytics. In addition, it generates detailed PDF reports and offers a secure, user-friendly platform—making financial management simpler, smarter, and more reliable.

1.2 Objectives

The Smart Financial Planning and Transaction Categorization system is built to achieve these objectives:

- **Secure Authentication:** Provide secure, reliable user authentication with Gmail-based OTP verification to safeguard privacy and protect data.
- **Record Keeping:** Help users easily record and track income, expenses, budgets, and savings goals in an organized way.

- **Categorization:** Automatically categorize transactions to bring clarity, deeper insights, and better spending analysis.
- **Visualization:** Deliver real-time dashboards with charts that show income trends, expense breakdowns, and overall spending behavior.
- **Budget Assessment:** Accurately assess budget health by comparing current spending against predefined limits.
- **Reporting:** Generate detailed PDF financial reports and let users share them directly via email.
- **System Integration:** Ensure smooth communication between the frontend and backend through Flask APIs and SQLite database management.

1.3 Methodology

The Smart Financial Planning and Transaction Categorization system is developed step by step with a clear, modular approach:

- **Requirement Analysis:** Define the key features such as OTP login, dashboards, transaction categorization, budget tracking, savings goals, and automatic report generation.
- **System Design:** Plan the overall setup with a responsive frontend (HTML, CSS, JavaScript), a Flask backend, and an SQLite database for reliable data handling.
- **Frontend Development:** Build simple, user-friendly pages for signup, login, adding income, tracking expenses, setting budgets, savings goals, and downloading reports.
- **Backend Development:** Create Flask APIs to manage OTP generation, email delivery, CRUD operations, financial calculations, and PDF creation.
- **Database Design:** Organize SQLite tables for users, incomes, expenses, budgets, and savings goals to keep data consistent and secure.
- **Integration:** Link all frontend pages with backend REST APIs and update dashboard charts instantly with live data.

- **Testing:** Run unit and integration tests for OTP login, authentication, CRUD functions, transaction categorization, budget checks, and PDF reports.
- **Deployment:** Launch the system locally or on a server, then refine the design and user experience for smooth performance and easy interaction.

1.4 Expected Outcomes

The system is expected to deliver:

- A secure, fully functional web app that makes personal finance management simple and reliable.
- An OTP-based login feature that adds an extra layer of safety to user accounts.
- Automatic categorization of income and expenses for clear, organized financial tracking.
- A real-time dashboard with charts, totals, and summaries to highlight spending patterns.
- Complete modules for tracking budgets and monitoring progress toward savings goals.
- A PDF report generator that users can easily download or share via email.
- A smooth, clean, and user-friendly interface that supports everyday financial planning.

1.5 Hardware and Software Requirements

Hardware Requirements

- Processor: Intel i3 or higher to run the system smoothly.
- RAM: Minimum 4 GB (8 GB recommended for faster performance).
- Storage: At least 512 MB of free disk space available.
- Internet Connection: Needed for OTP verification and email services.

Software Requirements

- Operating System: Compatible with Windows, Linux, or macOS.

- Programming Language: Python 3.x.
- Framework: Flask for backend development.
- Frontend Tools: HTML, CSS, and JavaScript.
- Database: SQLite to securely store user and financial records.
- Email Service: SMTP (Gmail) for sending OTPs and reports.
- Editor/IDE: VS Code, PyCharm, or any preferred coding tool.
- Web Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox.

CHAPTER 2

FUNDAMENTALS OF PYTHON



2.1 Introduction to Python

Python is a versatile, high-level programming language praised for its clean syntax, readability, and ease of use. Its design encourages simple, well-structured code, making it suitable for everything from quick scripts to large-scale applications. With dynamic typing and automatic memory handling, Python reduces complexity and has become a popular choice for web development, automation, and data processing.

In the Smart Financial Planning system, Python powers the backend and manages essential tasks. It handles secure user authentication, OTP generation and email delivery, database communication with SQLite, transaction processing, and financial calculations. Using Flask—a lightweight Python framework—the system ensures fast API responses, smooth routing, and reliable performance.

2.2 Advantages of Python

Python brings several key advantages that make it a strong foundation for this financial planning system:

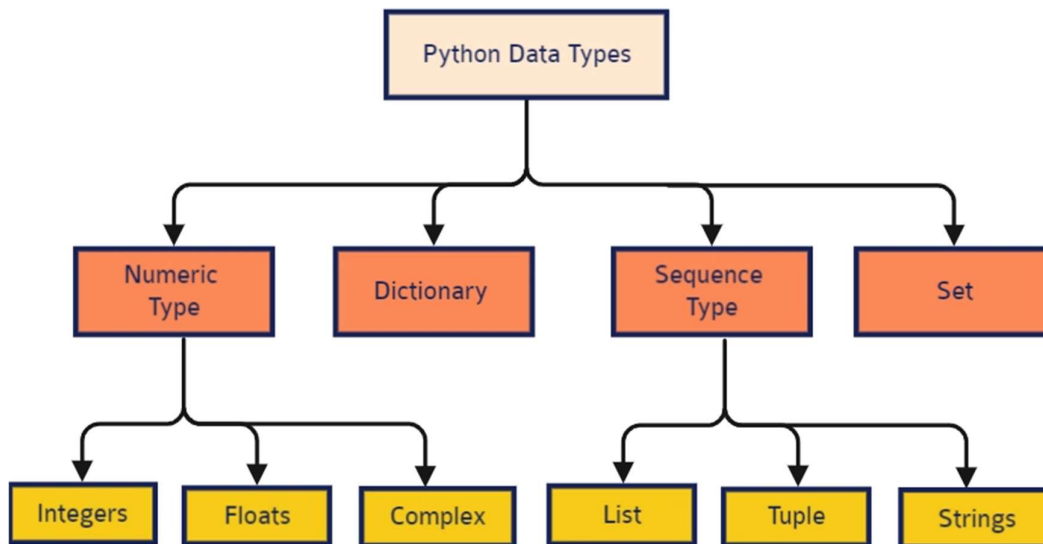
- **Readable and Maintainable Code:** Its clear, simple syntax makes backend tasks like OTP generation and expense tracking easy to build and maintain.
- **Extensive Standard Library:** Modules such as `sqlite3`, `smtplib`, `random`, and `datetime` provide essential features without needing extra tools.

- **Rich Library Ecosystem:** Frameworks like Flask for server routing and ReportLab for PDF creation speed up development.
- **Cross-Platform Compatibility:** Python runs smoothly on Windows, macOS, and Linux.
- **Efficient Database Handling:** It works seamlessly with SQLite, ensuring smooth storage and management of users and financial records.

2.3 Data Types

Used in the Project Python's built-in data types play a key role in backend logic.

- **Numeric Types (int, float):** Used for transaction amounts, budgets, and calculations.
- **Sequence Types (str, list, tuple):** Strings store emails and categories; lists hold database rows.
- **Mapping Type (dict):** Used for JSON-like structures between backend and frontend.
- **Boolean Type (bool):** Used in login validation and OTP matching.



2.4 Python Numbers

Python supports three numeric types, each serving different needs. Integers (int) are used for OTPs, IDs, and category totals. Floating-Point Numbers (float) are used for precise calculations like percentages, savings, and summaries.

2.5 Python Strings

Strings are widely used in the system for storing user email addresses, sending OTP codes, naming categories (Food, Travel, Bills), and writing SQL queries . Common string operations include `.strip()` for cleaning inputs and `.lower()` for consistent comparisons.

2.6 Python Lists

Lists are essential because the application often works with groups of data, such as income entries fetched from the database, expenses categorized by type, and chart datasets for dashboards .

2.7 Python Tuples

Tuples are used when values need to remain fixed, such as returning multiple values from functions or handling rows returned by SQLite, which come in tuple format .

2.8 Python Sets

Sets are useful for managing unique items, such as extracting distinct transaction categories and checking if a category already exists before adding it .

2.9 Python Dictionaries

Dictionaries are central to the project's structure. They are used for storing dashboard metrics (e.g., `"income": total_income`), managing JSON-like responses, and structuring data for PDF reports .

2.10 Functions in Python

Functions form the backbone of the backend. Key functions include:

- **OTP Generation Function:** Creates a random 4-digit number.
- **Email Sending Function:** Uses `smtplib` to deliver OTPs or PDF reports.
- **Database Functions:** Handle insert, update, delete, and fetch operations.
- **Report Generation Function:** Formats financial data into a PDF file.

CHAPTER 3

FUNDAMENTALS OF DBMS

3.1 Introduction

A Database Management System (DBMS) is software built to store, organize, and manage data in a structured way. In practice, the DBMS acts as a bridge between the backend program and the stored data. For this project, SQLite is used. SQLite is a lightweight, serverless relational database system widely used in web applications. It stores financial information such as user accounts, income entries, expenses, budgets, savings goals, and reports.

3.2 Characteristics of a DBMS SQLite

provides features suitable for financial applications:

- **Real-World Entity Representation:** Entities such as User, Income, and Expense are represented as tables.
- **Relation-Based Tables:** Data is stored in structured tables with relationships (e.g., each expense belongs to a specific user).
- **ACID Properties:** Transactions follow Atomicity, Consistency, Isolation, and Durability, ensuring reliability.
- **Security:** By storing hashed passwords and separating OTP validation, the DBMS helps protect sensitive financial information.

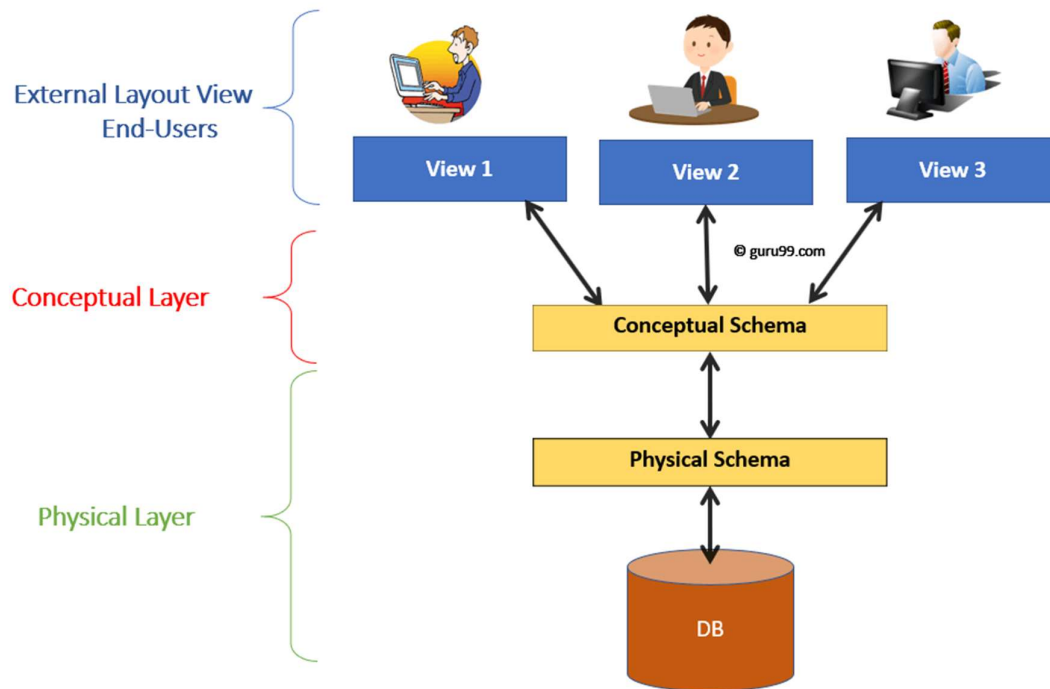
3.3 Data Model

This project uses the Relational Model, where data is stored in row-column tables. Each record is a row, and each field (amount, date, category) is a column. The Entity-Relationship (ER) Model is used to design the logical structure, identifying entities (User, Income, Budget) and their attributes.

3.4 Three-Schema Architecture

- **Internal Level:** How SQLite stores data internally on disk.
- **Conceptual Level:** Represents the overall database structure (tables and relationships).

- **External Level:** Shows only the data needed for a specific user (e.g., the dashboard displays only the logged-in user's data).



3.5 DBMS Component Modules

- **Query Processor:** Interprets DDL (creation of tables) and DML (inserting/updating data).
- **Storage Manager:** Includes the Authorization Manager (ensures only authenticated users access data) and Transaction Manager (guarantees safe additions of income/expenses).

3.6 Entity–Relationship (ER) Model

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

- Entities: User, Income, Expense, Budget, Savings Goal .
- Relationships: One user can have multiple income entries, expenses, budgets, and savings goals .

3.7 Relational Schema

The relational schema defines the structure of the database tables:

- Users: id (PK), email, password_hash, verified .
- Income: id (PK), user_id (FK), amount, source, date .
- Expense: id (PK), user_id (FK), amount, category, description, date .
- Budget: id (PK), user_id (FK), category, limit_amount .
- SavingsGoals: id (PK), user_id (FK), target_amount, saved_amount, deadline .

CHAPTER 4

FUNDAMENTALS OF SQL

4.1 Introduction

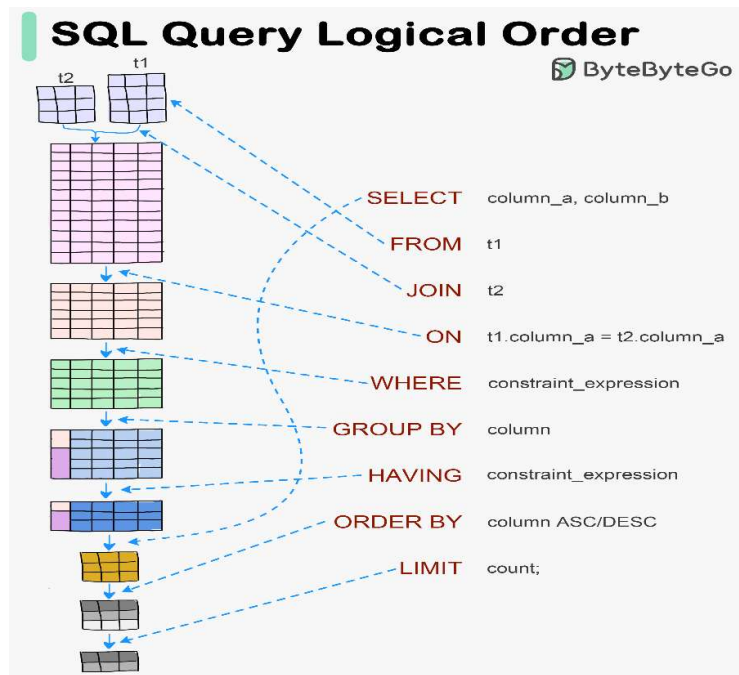
Structured Query Language (SQL) is the universal language used to manage and manipulate data in relational databases. It provides a reliable way to store, retrieve, update, and delete information efficiently.

In the Smart Financial Planning and Transaction Categorization system, SQLite is the chosen relational database. While the application interacts with the database through Python's module in Flask, every operation is ultimately translated into SQL queries that run on SQLite.

A clear understanding of SQL is crucial for this project because it supports table design, user data management, income and expense tracking, budget calculations, and the generation of accurate financial reports.

4.2 SQL Commands

SQL commands are grouped into five categories: DDL (Data Definition), DML (Data Manipulation), DCL (Data Control), TCL (Transaction Control), and DQL (Data Query).



4.3 Data Definition Language (DDL)

DDL commands define the structure of the database.

- **CREATE:** Used to create tables like users, income, and budgets.

CREATE

CREATE TABLE users (

```
id INTEGER PRIMARY KEY,  
name TEXT,  
email TEXT UNIQUE,  
password TEXT,  
verified INTEGER  
);
```

- **ALTER:** Used to modify existing tables, such as adding a deadline column to goals.

ALTER

```
ALTER TABLE expenses ADD COLUMN description TEXT;
```

- **DROP:** Deletes an entire table (used during development).

4.4 Data Manipulation Language (DML)

DML commands handle inserting, updating, and deleting data.

- **INSERT:** Adds new records (e.g., registering new users, adding expenses).

INSERT

```
INSERT INTO income (user_id, amount, source, date)
```

```
VALUES (1, 50000, 'Salary', '2025-01-01');
```

- **UPDATE:** Modifies existing records (e.g., updating saved amounts in goals).

UPDATE

```
UPDATE users SET verified = 1 WHERE email = 'user@gmail.com';
```

- **DELETE:** Removes specific rows (e.g., deleting incorrect records).

DELETE

```
DELETE FROM expenses WHERE id = 3;
```

4.5 Data Control Language (DCL)

DCL commands manage access and permissions in a database.

- **GRANT**

- **REVOKE**

SQLite is used locally, and access control is handled at the application level through authentication and OTP verification rather than database-level permissions. As a result, DCL commands are not explicitly used.

4.6 Transaction Control Language (TCL)

TCL commands ensure safe and consistent database operations.

- **COMMIT:** Saves all changes permanently. Used automatically after successful insert, update, or delete operations.
- **ROLLBACK:** Reverts changes if an error occurs during a transaction.
- **SAVEPOINT:** Creates checkpoints within a transaction (not explicitly used here).

In Flask with SQLite, transactions are typically handled using:

db.commit()

4.7 Data Query Language (DQL)

DQL is used to retrieve data from the database, primarily through the SELECT command. This is essential for dashboards, reports, and summaries.

Examples from the project:

- **Login Validation:**

```
SELECT * FROM users WHERE email = 'user@gmail.com';
```

- **Dashboard Data (Income & Expenses):**

```
SELECT * FROM income WHERE user_id = 1;
```

```
SELECT * FROM expenses WHERE user_id = 1;
```

- **Budget Calculation:**

```
SELECT * FROM budgets WHERE user_id = 1;
```

- **Savings Goals Display:**

```
SELECT * FROM savingsgoals WHERE user_id = 1;
```

These queries ensure that each user only sees their own financial data, maintaining privacy and security across the system.

CHAPTER 5

SYSTEM DESIGN

5.1 Creating the Database

The database was set up first to keep financial data safe and accessible. SQLite was chosen for its lightweight nature. Tables were created for users, income, expenses, budgets, and savings goals. Python and Flask create these tables automatically when the app starts, ensuring all fields are ready.

5.2 Connecting the Database

Connecting the database to the Application Flask connects to SQLite using Python's built-in `sqlite3` module. A helper function `get_db()` opens a new connection for each request and closes it after use. Sensitive details like the Flask secret key and SMTP credentials are stored safely in environment files.

5.3 Implementing the Main Web Interface

The "Main Window" is the Dashboard that appears after login. The `/dashboard` route loads the homepage, built with HTML, CSS (Tailwind), and JavaScript. It features a responsive design that looks clean on mobile devices and desktops, with a navigation bar linking to all modules.

5.4 Displaying Different Modules in the Main Window

- Income Module: Shows income entries in table rows.
- Expense Module: Displays categorized expenses with delete options.
- Budget Module: Lets users create budgets and view them as cards.
- Financial Dashboard: Uses `Chart.js` to render bar and pie charts for financial insights.

5.5 Processing Queries Flask handles backend logic through RESTful API routes

When users add data, the frontend sends POST requests to routes like `/api/income`. Python validates the fields, generates OTPs, calculates budget usage, and returns JSON data so the frontend can update instantly.

5.6 Generating Financial Reports

`ReportLab` and `Matplotlib` are used to create financial reports. The system generates a structured PDF with transactions and summaries. `Matplotlib` creates pie charts to show spending by category. Users can download this PDF or have it emailed via SMTP.

CHAPTER 6

IMPLEMENTATION

6.1 Overview

The Smart Budget Tracking System is designed to make managing money simple and secure. It focuses on secure user authentication via Gmail-based OTP, organized financial tracking, automatic categorization, and visual insights.

6.2 Database Structure

The backend uses SQLite with the following structure:

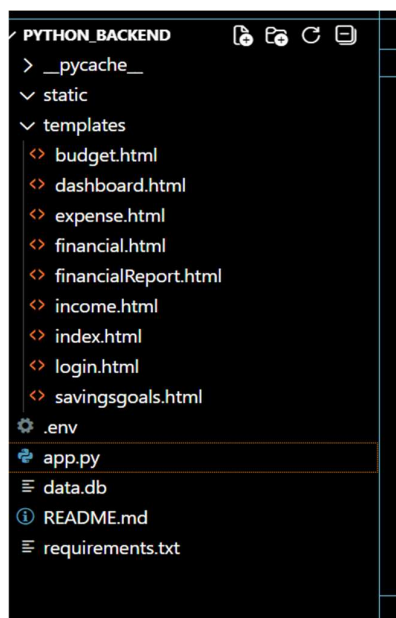
- **Users Table:** Stores account details and verification status.
- **Income Table:** Records source, amount, and date.
- **Expenses Table:** Records amount, category, description, and date.
- **Budgets Table:** Stores category-specific limits.
- **Savings Goals Table:** Stores targets, saved amounts, and deadlines.

6.3 GUI Structure

The GUI is built with HTML, Tailwind CSS, and JavaScript.

- **Login/Register:** Simple forms for authentication.
- **Dashboard:** A central hub showing totals, graphs, and recent transactions.
- **Entry Pages:** Forms for adding income, expenses, and goals.
- **Report Page:** Options to download or email PDF reports.

6.4 Code Structure



6.4.1 Backend Implementation

File: app.py (Main Application Logic) This file handles database connections, user authentication (Login/Register/OTP), and all financial API endpoints (Income, Expense, Budget, Goals).

App.py

PART 1 of 3

app.py — PART 1: DB, Auth (register/login/verify), income & expense APIs

```
import os
```

```
import sqlite3
```

```
import io
```

```
from flask import Flask, render_template, request, jsonify, g, send_file
```

```
from werkzeug.security import generate_password_hash, check_password_hash
```

```
from dotenv import load_dotenv
```

```
import secrets
```

```
import random
```

```
import smtplib
```

```
from email.message import EmailMessage
```

```
from datetime import datetime, timedelta
```

```
from flask_cors import CORS
```

```
# Optional PDF library (used later). Presence check only.
```

```
try:
```

```
    from reportlab.pdfgen import canvas
```

```
    from reportlab.lib.pagesizes import letter
```

```
except Exception:
```

```
    canvas = None
```

```
    letter = None
```

```
# Load .env
```

```
load_dotenv()
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
DB_PATH = os.path.join(BASE_DIR, "data.db")
```

```
app = Flask(__name__, static_folder="static", template_folder="templates")
```

```
CORS(app)
```

```
app.config["SECRET_KEY"] = os.getenv("FLASK_SECRET", secrets.token_hex(16))
```

```
# In-memory OTP store (simple)
```



```
otp_store = {}

# ----- DB CONNECTION & MIGRATION HELPERS -----

def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH, timeout=5, check_same_thread=False)
        g.db.row_factory = sqlite3.Row
    return g.db

@app.teardown_appcontext
def close_db(exception):
    db = g.pop("db", None)
    if db:
        db.close()

def column_exists(cursor, table_name, column_name):
    cursor.execute(f"PRAGMA table_info({table_name})")
    cols = [row["name"] for row in cursor.fetchall()]
    return column_name in cols

def init_db():
    db = get_db()
    cur = db.cursor()
    # users
    cur.execute("""
        CREATE TABLE IF NOT EXISTS users(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            email TEXT UNIQUE,
            password TEXT,
            verified INTEGER DEFAULT 0
        )
    """)
    # income
    cur.execute("""
        CREATE TABLE IF NOT EXISTS income(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
        user_id INTEGER,
        amount REAL,
        source TEXT,
        date TEXT
    )
    """)
# expenses
cur.execute("""
    CREATE TABLE IF NOT EXISTS expenses(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER,
        amount REAL,
        category TEXT,
        date TEXT
    )
    """)
if not column_exists(cur, "expenses", "description"):
    try:
        cur.execute("ALTER TABLE expenses ADD COLUMN description TEXT")
    except:
        pass
# budgets
cur.execute("""
    CREATE TABLE IF NOT EXISTS budgets(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER,
        category TEXT,
        amount REAL
    )
    """)
# savingsgoals
cur.execute("""
    CREATE TABLE IF NOT EXISTS savingsgoals(
```

```
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER,
        goalName TEXT,
        targetAmount REAL,
        savedAmount REAL DEFAULT 0,
        deadline TEXT
    )
    """
# MIGRATIONS
if not column_exists(cur, "savingsgoals", "savedAmount"):
    try:
        cur.execute("ALTER TABLE savingsgoals ADD COLUMN savedAmount REAL
DEFAULT 0")
    except Exception:
        pass
if not column_exists(cur, "savingsgoals", "deadline"):
    try:
        cur.execute("ALTER TABLE savingsgoals ADD COLUMN deadline TEXT")
        print("Added missing column: deadline")
    except Exception as e:
        print("Deadline MIGRATION ERROR:", e)
db.commit()
# initialize DB on app start (will run at bottom main)
# ----- EMAIL / OTP helper -----
def send_email_otp(to_email, otp):
    smtp_host = os.getenv("SMTP_HOST")
    smtp_port = int(os.getenv("SMTP_PORT", 587))
    smtp_user = os.getenv("SMTP_USER")
    smtp_pass = os.getenv("SMTP_PASS")
    print("\n===== SENDING OTP EMAIL =====")
    print("SMTP HOST :", smtp_host)
    print("SMTP PORT :", smtp_port)
    print("SMTP USER :", smtp_user)
    print("=====\\n")
```

```
if not smtp_host or not smtp_user or not smtp_pass:
    print("EMAIL ERROR: SMTP settings missing")
    return False

msg = EmailMessage()
msg["Subject"] = "Your Budget Tracker OTP"
msg["From"] = smtp_user
msg["To"] = to_email
msg.set_content(f"Your OTP is: {otp}\nValid for 10 minutes.")
try:
    server = smtplib.SMTP(smtp_host, smtp_port, timeout=10)
    server.starttls()
    print("Connecting to SMTP server...")
    server.login(smtp_user, smtp_pass)
    print("SMTP Login successful!")
    server.send_message(msg)
    server.quit()
    print("OTP Email sent successfully!")
    return True
except Exception as e:
    print("EMAIL ERROR:", e)
    return False

# ----- TEMPLATE ROUTES -----
@app.route("/")
def index():
    return render_template("login.html")

@app.route("/login")
def login_page():
    return render_template("login.html")

@app.route("/dashboard")
def dashboard_page():
    return render_template("dashboard.html")

@app.route("/income")
def income_page():
```

```
    return render_template("income.html")
@app.route("/expense")
def expense_page():
    return render_template("expense.html")
@app.route("/budget")
def budget_page():
    return render_template("budget.html")
@app.route("/savingsgoals")
def savings_page():
    return render_template("savingsgoals.html")
@app.route("/financialreport")
def financialreport_page():
    return render_template("financialreport.html")
@app.route("/financial")
def financial_summary_page():
    return render_template("financial.html")
# ----- SMALL HELPERS -----
def safe_int(v):
    try:
        return int(v)
    except Exception:
        return None
def safe_float(v):
    try:
        return float(v)
    except Exception:
        return None
def get_userid_from_payload(data):
    uid = data.get("user_id")
    if uid is None:
        uid = data.get("userId")
    try:
        return int(uid) if uid is not None else None
```

```
except Exception:
```

```
    return None
```

```
# ----- REGISTER -----
```

```
@app.route("/api/register", methods=["POST"])
```

```
def register():
```

```
    data = request.get_json() or {}
```

```
    name = (data.get("name") or "").strip()
```

```
    email = (data.get("email") or "").strip().lower()
```

```
    password = data.get("password")
```

```
    if not (name and email and password):
```

```
        return jsonify({"error": "Missing fields"}), 400
```

```
    db = get_db()
```

```
    cur = db.cursor()
```

```
    cur.execute("SELECT * FROM users WHERE email = ?", (email,))
```

```
    existing = cur.fetchone()
```

```
    if existing and existing["verified"]:
```

```
        return jsonify({"error": "Email already registered"}), 400
```

```
    otp = f'{random.randint(1000, 9999)}'
```

```
    otp_store[email] = {
```

```
        "code": otp,
```

```
        "expires": datetime.utcnow() + timedelta(minutes=10)
```

```
    }
```

```
    sent = send_email_otp(email, otp)
```

```
    if not sent:
```

```
        return jsonify({"error": "Failed to send OTP. Check SMTP settings."}), 500
```

```
    if not existing:
```

```
        cur.execute(
```

```
            "INSERT INTO users(name, email, password, verified) VALUES (?, ?, ?, ?)",
```

```
        (name, email, generate_password_hash(password), 0)
    )
    db.commit()
    return jsonify({"message": "OTP sent"}), 200
# ----- VERIFY -----
@app.route("/api/verify", methods=["POST"])
def verify():
    data = request.get_json() or {}
    email = (data.get("email") or "").lower()
    code = str(data.get("code") or "")
    otp_data = otp_store.get(email)
    if not otp_data:
        return jsonify({"error": "No OTP found"}), 400
    if datetime.utcnow() > otp_data["expires"]:
        otp_store.pop(email, None)
        return jsonify({"error": "OTP expired"}), 400
    if otp_data["code"] != code:
        return jsonify({"error": "Invalid OTP"}), 400
    db = get_db()
    cur = db.cursor()
    cur.execute("UPDATE users SET verified = 1 WHERE email = ?", (email,))
    db.commit()
    otp_store.pop(email, None)
    return jsonify({"message": "Verified"}), 200
# ----- LOGIN -----
@app.route("/api/login", methods=["POST"])
def login():
    data = request.get_json() or {}
    email = (data.get("email") or "").lower()
    password = data.get("password")
    db = get_db()
    cur = db.cursor()
    cur.execute("SELECT * FROM users WHERE email = ?", (email,))
```

```
user = cur.fetchone()
if not user:
    return jsonify({"error": "User not found"}), 404
if not user["verified"]:
    return jsonify({"error": "Email not verified"}), 403
if not check_password_hash(user["password"], password):
    return jsonify({"error": "Invalid password"}), 401
    return jsonify({"user": {"id": user["id"], "name": user["name"], "email":
user["email"]}}), 200
# ----- INCOME API -----
@app.route("/api/income", methods=["POST"])
def add_income():
    data = request.get_json() or {}
    user_id = get_userid_from_payload(data)
    amount = safe_float(data.get("amount"))
    source = (data.get("source") or "").strip()
    date = data.get("date") or datetime.utcnow().strftime("%Y-%m-%d")
    if not all([user_id, amount is not None, source, date]):
        return jsonify({"error": "Missing or invalid fields"}), 400
    db = get_db()
    cur = db.cursor()
    cur.execute("INSERT INTO income (user_id, amount, source, date) VALUES
(?, ?, ?, ?)",
                (user_id, amount, source, date))
    db.commit()
    new_id = cur.lastrowid
    cur.execute("SELECT * FROM income WHERE id = ?", (new_id,))
    created = dict(cur.fetchone())
    return jsonify({"message": "Income added", "income": created}), 201
@app.route("/api/income/user/<int:user_id>", methods=["GET"])
def get_income(user_id):
    db = get_db()
    cur = db.cursor()
```



```
cur.execute("SELECT * FROM income WHERE user_id = ? ORDER BY date DESC,
id DESC", (user_id,))
rows = [dict(row) for row in cur.fetchall()]
return jsonify(rows), 200

@app.route("/api/income/<int:income_id>", methods=["DELETE"])
def delete_income(income_id):
    db = get_db()
    cur = db.cursor()
    cur.execute("DELETE FROM income WHERE id = ?", (income_id,))
    db.commit()
    return jsonify({"message": "Income deleted"}), 200

# ----- EXPENSE API -----

@app.route("/api/expenses", methods=["POST"])
def add_expense():
    data = request.get_json() or {}
    user_id = get_userid_from_payload(data)
    amount = safe_float(data.get("amount"))
    category = (data.get("category") or "").strip()
    description = (data.get("description") or "").strip()
    date = data.get("date") or datetime.utcnow().strftime("%Y-%m-%d")
    if not all([user_id, amount is not None, category, date]):
        return jsonify({"error": "Missing or invalid fields"}), 400
    db = get_db()
    cur = db.cursor()
    # ensure column exists (migrate if needed)
    if not column_exists(cur, "expenses", "description"):
        try:
            cur.execute("ALTER TABLE expenses ADD COLUMN description TEXT")
        except Exception:
            pass
    cur.execute("INSERT INTO expenses (user_id, amount, category, description, date)
VALUES (?, ?, ?, ?, ?)",
                (user_id, amount, category, description, date))
    db.commit()
```

```
new_id = cur.lastrowid
cur.execute("SELECT * FROM expenses WHERE id = ?", (new_id,))
created = dict(cur.fetchone())
return jsonify({"message": "Expense added", "expense": created}), 201

@app.route("/api/expenses/user/<int:user_id>", methods=["GET"])
def get_expenses(user_id):
    db = get_db()
    cur = db.cursor()
    cur.execute("SELECT * FROM expenses WHERE user_id = ? ORDER BY date DESC,
id DESC", (user_id,))
    rows = [dict(row) for row in cur.fetchall()]
    return jsonify(rows), 200

@app.route("/api/expenses/<int:expense_id>", methods=["DELETE"])
def delete_expense(expense_id):
    db = get_db()
    cur = db.cursor()
    cur.execute("DELETE FROM expenses WHERE id = ?", (expense_id,))
    db.commit()
    return jsonify({"message": "Expense deleted"}), 200

# PART 1 END (I'll send Part 2 next)
# =====
# PART 2 — Budgets + Savings Goals + Financial Summary
# =====
# ----- BUDGETS API -----

@app.route("/api/budgets", methods=["POST"])
def add_budget():
    data = request.get_json() or {}
    user_id = get_userid_from_payload(data)
    category = (data.get("category") or "").strip()
    amount = safe_float(data.get("amount"))
    if not all([user_id, category, amount is not None]):
        return jsonify({"error": "Missing or invalid fields"}), 400
    db = get_db()
    cur = db.cursor()
```

```
cur.execute("""
    INSERT INTO budgets (user_id, category, amount)
    VALUES (?, ?, ?)
""", (user_id, category, amount))
db.commit()
new_id = cur.lastrowid
cur.execute("SELECT * FROM budgets WHERE id = ?", (new_id,))
created = dict(cur.fetchone())
return jsonify({"message": "Budget added", "budget": created}), 201

@app.route("/api/budgets/user/<int:user_id>", methods=["GET"])
def get_budgets(user_id):
    db = get_db()
    cur = db.cursor()
    cur.execute("SELECT * FROM budgets WHERE user_id = ?", (user_id,))
    rows = [dict(row) for row in cur.fetchall()]
    return jsonify(rows), 200

@app.route("/api/budgets/<int:budget_id>", methods=["DELETE"])
def delete_budget(budget_id):
    db = get_db()
    cur = db.cursor()

    cur.execute("DELETE FROM budgets WHERE id = ?", (budget_id,))
    db.commit()
    return jsonify({"message": "Budget deleted"}), 200

# ----- SAVINGS GOALS API -----

@app.route("/api/savingsgoals", methods=["POST"])
def add_saving_goal():
    data = request.get_json() or {}
    user_id = get_userid_from_payload(data)
    goalName = (data.get("goalName") or "").strip()
    targetAmount = safe_float(data.get("targetAmount"))
    savedAmount = safe_float(data.get("savedAmount")) or 0
    deadline = data.get("deadline") or
```

```
if not all([user_id, goalName, targetAmount is not None]):
    return jsonify({"error": "Missing or invalid fields"}), 400
db = get_db()
cur = db.cursor()
cur.execute("""
    INSERT INTO savingsgoals (user_id, goalName, targetAmount, savedAmount,
    deadline)
    VALUES (?, ?, ?, ?, ?)
    """, (user_id, goalName, targetAmount, savedAmount, deadline))
db.commit()
new_id = cur.lastrowid
cur.execute("SELECT * FROM savingsgoals WHERE id = ?", (new_id,))
created = dict(cur.fetchone())
return jsonify({"message": "Goal added", "goal": created}), 200
@app.route("/api/savingsgoals/user/<int:user_id>", methods=["GET"])
def get_saving_goals(user_id):
    db = get_db()
    cur = db.cursor()
    cur.execute("SELECT * FROM savingsgoals WHERE user_id = ?", (user_id,))
    rows = [dict(row) for row in cur.fetchall()]
    return jsonify(rows), 200
@app.route("/api/savingsgoals/<int:goal_id>", methods=["DELETE"])
def delete_saving_goal(goal_id):
    db = get_db()
    cur = db.cursor()
    cur.execute("DELETE FROM savingsgoals WHERE id = ?", (goal_id,))
    db.commit()
    return jsonify({"message": "Goal deleted"}), 200
# ----- FINANCIAL SUMMARY (Dashboard) -----
@app.route("/api/financials/history/<int:user_id>", methods=["GET"])
def financials_history(user_id):
    db = get_db()
    cur = db.cursor()
    # income
```

```
cur.execute("SELECT * FROM income WHERE user_id = ?", (user_id,))
incomes = [dict(x) for x in cur.fetchall()]
# expenses
cur.execute("SELECT * FROM expenses WHERE user_id = ?", (user_id,))
expenses = [dict(x) for x in cur.fetchall()]
# budgets
cur.execute("SELECT * FROM budgets WHERE user_id = ?", (user_id,))
budgets = [dict(x) for x in cur.fetchall()]
# saving goals
cur.execute("SELECT * FROM savingsgoals WHERE user_id = ?", (user_id,))
goals = [dict(x) for x in cur.fetchall()]
total_income = sum(float(i["amount"]) for i in incomes)
total_expense = sum(float(e["amount"]) for e in expenses)
total_budgets = sum(float(b["amount"]) for b in budgets)
total_savings = sum(float(g["savedAmount"]) for g in goals)
remaining = total_income - total_expense - total_savings
return jsonify({
    "incomes": incomes,
    "expenses": expenses,
    "budgets": budgets,
    "goals": goals,
    "totals": {
        "totalIncome": total_income,
        "totalExpense": total_expense,
        "totalBudget": total_budgets,
        "totalSavings": total_savings,
        "remainingBalance": remaining
    }
}), 200# =====
# =====
# PART 3 — Premium PDF Report with Tables + Clean Pie Chart
# =====
from reportlab.pdfgen import canvas
```

```
from reportlab.lib.pagesizes import letter
from reportlab.lib.utils import ImageReader
from reportlab.lib import colors
from reportlab.platypus import Table, TableStyle
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import io
from datetime import datetime, timezone

# ----- PIE CHART GENERATOR (with legend + separated slices) -----
def generate_expense_pie(expenses):
    if not expenses:
        return None
    # Aggregate categories
    categories = {}
    for e in expenses:
        try:
            amt = float(e.get("amount", 0))
        except:
            amt = 0
        categories[e.get("category", "Other")] = categories.get(e.get("category", "Other"), 0)
    + amt
    if sum(categories.values()) == 0:
        return None
    # Colors
    color_palette = [
        "#6366F1", "#EC4899", "#10B981", "#F59E0B",
        "#3B82F6", "#8B5CF6", "#EF4444", "#14B8A6"
    ]
    labels = list(categories.keys())
    values = list(categories.values())
    explode = [0.04] * len(values)
    # Separate slices slightly
    fig, ax = plt.subplots(figsize=(5.2, 4.2), dpi=120)
```

```
wedges, texts, autotexts = ax.pie(
    values,
    labels=None,          # no inline labels
    autopct="%1.1f%%",
    explode=explode,
    colors=color_palette[:len(values)],
    pctdistance=0.8,
    labeldistance=1.1,
    startangle=120
)
ax.set_title("Expense Breakdown", fontsize=14, fontweight="bold")
# Add legend outside
ax.legend(wedges, labels, title="Categories", loc="center left", bbox_to_anchor=(1,
0.5))
ax.axis("equal")
buf = io.BytesIO()
plt.tight_layout()
plt.savefig(buf, format="png", bbox_inches="tight")
plt.close(fig)
buf.seek(0)
return buf

# ----- SECTION TITLE -----
def draw_title(pdf, title, y):
    pdf.setFont("Helvetica-Bold", 14)
    pdf.setFillColors(colors.HexColor("#4F46E5"))
    pdf.drawString(50, y, title)
    return y - 25

# ----- TABLE BUILDER -----
def draw_table(pdf, data_list, columns, y_pos):
    """
    data_list: list of dicts
    columns: [ ("Label", "dictKey"), ... ]
    """
    if not data_list:
```

```
pdf.setFont("Helvetica", 10)
pdf.drawString(60, y_pos, "No records available.")
return y_pos - 20

# Build header + rows
table_data = [[col[0] for col in columns]]
for row in data_list:
    table_data.append([str(row.get(col[1], "")) for col in columns])
table = Table(table_data, colWidths=[120, 120, 120, 120])
table.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#6366F1")),
    ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
    ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
    ("FONTSIZE", (0, 0), (-1, -1), 9),
    ("ALIGN", (0, 0), (-1, -1), "LEFT"),
    ("BACKGROUND", (0, 1), (-1, -1), colors.HexColor("#F3F4F6")),
    ("ROWBACKGROUNDS", (0, 1), (-1, -1),
        [colors.white, colors.HexColor("#F3F4F6")]),
    ("BOX", (0, 0), (-1, -1), 1, colors.HexColor("#D1D5DB")),
    ("GRID", (0, 0), (-1, -1), 0.5, colors.gray)
]))
table.wrapOn(pdf, 50, y_pos)
table.drawOn(pdf, 50, y_pos - (20 * len(table_data)))
return y_pos - (22 * len(table_data)) - 10

# ----- MAIN PREMIUM PDF -----
def generate_financial_pdf(user_id):
    db = get_db()
    cur = db.cursor()
    # Fetch user
    cur.execute("SELECT name FROM users WHERE id=?", (user_id,))
    username = (cur.fetchone() or {"name": "User"})["name"]
    # Fetch all data
    cur.execute("SELECT * FROM income WHERE user_id=?", (user_id,))
    incomes = [dict(i) for i in cur.fetchall()]
```



```

cur.execute("SELECT * FROM expenses WHERE user_id=?", (user_id,))
expenses = [dict(e) for e in cur.fetchall()]
cur.execute("SELECT * FROM budgets WHERE user_id=?", (user_id,))
budgets = [dict(b) for b in cur.fetchall()]
cur.execute("SELECT * FROM savingsgoals WHERE user_id=?", (user_id,))
goals = [dict(g) for g in cur.fetchall()]

# Totals
total_income = sum(float(i["amount"]) for i in incomes)
total_expense = sum(float(e["amount"]) for e in expenses)
total_budget = sum(float(b["amount"]) for b in budgets)
total_saved = sum(float(g["savedAmount"]) for g in goals)
balance = total_income - total_expense

# PDF buffer
buffer = io.BytesIO()
pdf = canvas.Canvas(buffer, pagesize=letter)
width, height = letter

# =====
# HEADER
# =====
pdf.setFillColor(colors.HexColor("#6366F1"))
pdf.rect(0, height - 110, width, 110, fill=1)
pdf.setFillColor(colors.white)
pdf.setFont("Helvetica-Bold", 26)
pdf.drawString(40, height - 60, "Financial Report")
pdf.setFont("Helvetica", 12)
pdf.drawString(40, height - 85, f"User: {username}")
pdf.drawString(250, height - 85,
               f"Generated: {datetime.now(timezone.utc).strftime('%Y-%m-%d %H:%M
UTC')}}")
y = height - 140
# =====
# SUMMARY
# =====
y = draw_title(pdf, "Summary Overview", y)

```

```

summary_rows = [
    ["Total Income", f"₹{total_income:.2f}"],
    ["Total Expenses", f"₹{total_expense:.2f}"],
    ["Total Budget", f"₹{total_budget:.2f}"],
    ["Total Saved", f"₹{total_saved:.2f}"],
    ["Remaining Balance", f"₹{balance:.2f}"]
]
table = Table(summary_rows, colWidths=[200, 200])
table.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, -1), colors.HexColor("#F3F4F6")),
    ("BOX", (0, 0), (-1, -1), 1, colors.gray),
    ("GRID", (0, 0), (-1, -1), 0.5, colors.gray),
    ("FONTNAME", (0, 0), (-1, -1), "Helvetica"),
    ("FONTSIZE", (0, 0), (-1, -1), 10),
    ("ALIGN", (0, 0), (-1, -1), "LEFT")
]))
table.wrapOn(pdf, 50, y)
table.drawOn(pdf, 50, y - 120)
y -= 150
# =====
# PIE CHART
# =====
pie_buf = generate_expense_pie(expenses)
if pie_buf:
    y = draw_title(pdf, "Expense Breakdown (Pie Chart)", y - 10)
    pdf.drawImage(ImageReader(pie_buf), 50, y - 280, width=380, height=280)
    y -= 300
# =====
# TRANSACTIONS TABLE
# =====
y = draw_title(pdf, "Recent Transactions", y - 10)
recent = []
for i in incomes[:6]:

```

```
        recent.append({"date": i["date"], "type": "Income", "item": i["source"], "amount":
f"+₹ {i['amount']}"})
    for e in expenses[:6]:
        recent.append({"date": e["date"], "type": "Expense", "item": e["category"], "amount":
f"-₹ {e['amount']}"})
    recent = sorted(recent, key=lambda x: x["date"], reverse=True)[:10]
    y = draw_table(pdf, recent, [
        ("Date", "date"),
        ("Type", "type"),
        ("Category / Source", "item"),
        ("Amount", "amount"),
    ], y)
    pdf.showPage()
    pdf.save()
    buffer.seek(0)
    return buffer

## =====
# SEND PDF REPORT BY EMAIL
# =====
@app.route("/api/report/email/<int:user_id>", methods=["POST"])
def email_report(user_id):
    try:
        # Get recipient email
        recipient = request.args.get("email")
        if not recipient:
            return jsonify({"error": "Missing email"}), 400
        # Generate PDF buffer
        pdf_buffer = generate_financial_pdf(user_id)
        pdf_bytes = pdf_buffer.getvalue()
        # Email settings
        smtp_host = os.getenv("SMTP_HOST")
        smtp_port = int(os.getenv("SMTP_PORT", 587))
        smtp_user = os.getenv("SMTP_USER")
        smtp_pass = os.getenv("SMTP_PASS")
```

```
    if not smtp_host or not smtp_user or not smtp_pass:
        return jsonify({"error": "SMTP not configured"}), 500
    # Build message
    msg = EmailMessage()
    msg["Subject"] = "Your Financial Report"
    msg["From"] = smtp_user
    msg["To"] = recipient
    msg.set_content("Attached is your financial report PDF.")
    # Attach PDF
    msg.add_attachment(
        pdf_bytes,
        maintype="application",
        subtype="pdf",
        filename=f"Financial_Report_{user_id}.pdf"
    )
    # Send email
    server = smtplib.SMTP(smtp_host, smtp_port)
    server.starttls()
    server.login(smtp_user, smtp_pass)
    server.send_message(msg)
    server.quit()
    return jsonify({"message": "Email sent successfully"}), 200
except Exception as e:
    print("EMAIL PDF ERROR:", e)
    return jsonify({"error": str(e)}), 500

# =====
# PDF DOWNLOAD ROUTE
# =====
# =====
# PDF DOWNLOAD ROUTE
# =====

@app.route("/api/report/download/<int:user_id>", methods=["GET"])
def download_report(user_id):
```

```
try:
    pdf_buffer = generate_financial_pdf(user_id)
    return send_file(
        pdf_buffer,
        mimetype="application/pdf",
        as_attachment=True,
        download_name=f'Financial_Report_{user_id}.pdf'
    )
except Exception as e:
    print("PDF DOWNLOAD ERROR:", e)
    return jsonify({"error": str(e)}), 500
# ----- START APP -----
if __name__ == "__main__":
    with app.app_context():
        init_db()    # Ensure DB tables + migrations run
        print("Starting Flask server... http://127.0.0.1:5000")
        app.run(debug=True)
```

Budget.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Budgets - Budget Tracker</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<script>
    function go(page) {
        window.location.href = page;
    }
</script>
<body class="bg-gray-100">
```

```

<!-- NAVBAR -->
<header class="bg-indigo-600 text-white py-4 shadow-md">
  <div class="max-w-5xl mx-auto px-4 flex justify-between items-center">
    <h1 class="text-2xl font-bold">Budgets</h1>
    <button onclick="go('/dashboard')"
      class="px-5 py-2 bg-indigo-600 text-white rounded-xl hover:bg-indigo-700
shadow-md">
      ← Back
    </button>
  </div>
</header>
<!-- MAIN CONTENT -->
<div class="max-w-6xl mx-auto px-4 py-8">
  <!-- ADD BUDGET -->
  <div class="bg-white p-8 shadow-lg rounded-2xl mb-10">
    <h2 class="text-xl font-bold text-gray-800 mb-4">+ Add New Budget</h2>
    <form id="budgetForm" class="grid grid-cols-1 md:grid-cols-2 gap-6">
      <div>
        <label class="block text-gray-600 font-medium mb-1">Category</label>
        <input id="category" type="text"
          class="w-full px-4 py-3 rounded-xl border border-gray-300 focus:ring-2
focus:ring-indigo-500"
          placeholder="e.g., Food, Travel" required>
      </div>
      <div>
        <label class="block text-gray-600 font-medium mb-1">Amount</label>
        <input id="amount" type="number"
          class="w-full px-4 py-3 rounded-xl border border-gray-300 focus:ring-2
focus:ring-indigo-500"
          placeholder="e.g., 20000" required>
      </div>
      <button type="submit"
        class="md:col-span-2 bg-indigo-600 hover:bg-indigo-700 text-white py-3
rounded-xl font-semibold shadow transition">

```

```

        Add Budget
      </button>
    </form>
  </div>
  <!-- EXISTING BUDGETS -->
  <div class="bg-white p-8 shadow-lg rounded-2xl">
    <h2 class="text-xl font-bold text-gray-800 mb-6"> 📅 Your Budgets</h2>
    <div id="budgetContainer" class="space-y-5">
      <!-- Budget cards injected here -->
    </div>
  </div>
</div>
<script>
  const API_BASE_URL = "http://localhost:5000/api/budgets";
  const userId = localStorage.getItem("userId");
  if (!userId) window.location.href = "login.html";
  async function loadBudgets() {
    const res = await fetch(`${API_BASE_URL}/user/${userId}`);
    const data = await res.json();
    const container = document.getElementById("budgetContainer");
    container.innerHTML = "";
    data.forEach(b => {
      const card = document.createElement("div");
      card.className =
        "p-6 bg-gray-50 rounded-xl border shadow-md flex justify-between items-
center";
      card.innerHTML = `
        <div>
          <h3 class="text-lg font-bold text-indigo-600">${b.category}</h3>
          <p class="text-gray-700 font-medium mt-1">Budget: ₹${b.amount}</p>
        </div>

        <button onclick="deleteBudget(${b.id})"

```

```
        class="bg-red-500 hover:bg-red-600 text-white px-4 py-2 rounded-xl shadow
transition">
            Delete
        </button>
        container.appendChild(card);
    });
}
document.getElementById("budgetForm").addEventListener("submit", async (e) =>
{
    e.preventDefault();
    const data = {
        userId: parseInt(userId),
        category: category.value,
        amount: parseFloat(amount.value)
    };
    await fetch(API_BASE_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(data)
    });
    budgetForm.reset();
    loadBudgets();
});
async function deleteBudget(id) {
    if (!confirm("Are you sure you want to delete this budget?")) return;
    await fetch(`${API_BASE_URL}/${id}`, { method: "DELETE" });
    loadBudgets();
}
function goBack() {
    window.location.href = "dashboard.html";
}
loadBudgets();
</script>
</body>
```


</html>

Dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Premium SaaS Dashboard — Budget Tracker</title>
  <!-- Tailwind (CDN) -->
  <script src="https://cdn.tailwindcss.com"></script>
  <!-- Chart.js -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>

  <style>
    /* Premium / Glass aesthetics */
    :root{
      --primary:#6366f1; /* indigo-500 */
      --primary-2:#8b5cf6; /* indigo-400 */
      --muted:#6b7280; /* gray-500 */
      --card-bg: rgba(255,255,255,0.85);
      --glass: rgba(255,255,255,0.38);
    }

    /* page background */
    body{
      background: linear-gradient(135deg, #f8fafc 0%, #eef2ff 35%, #f5f3ff 100%);
      -webkit-font-smoothing:antialiased;
      font-family: Inter, ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto,
      "Helvetica Neue", Arial;
    }

    /* Glass navbar */
```

```
.glass-nav{
  backdrop-filter: blur(10px);
  background: linear-gradient(90deg, rgba(255,255,255,0.40), rgba(255,255,255,0.20));
  border-bottom: 1px solid rgba(99,102,241,0.08);
}

/* nav button style */
.nav-btn{
  display:inline-flex;
  align-items:center;
  gap:.6rem;
  padding:.6rem .9rem;
  border-radius: .75rem;
  font-weight: 600;
  color: white;
  background: linear-gradient(90deg, var(--primary), var(--primary-2));
  box-shadow: 0 6px 18px rgba(99,102,241,0.14);
  transition: transform .18s ease, box-shadow .18s ease;
}
.nav-btn:hover{ transform: translateY(-3px); box-shadow: 0 10px 30px
rgba(99,102,241,0.18); }

/* small premium card */
.premium-card{
  background: var(--card-bg);
  border-radius: 1rem;
  padding: 1.25rem;
  box-shadow: 0 8px 30px rgba(15,23,42,0.06);
  position: relative;
  overflow: hidden;
  border: 1px solid rgba(99,102,241,0.05);
}
.premium-card .kpi { font-size:1.5rem; font-weight:700; color: #111827; }
.kpi-sub { color:var(--muted); font-weight:600; font-size:.9rem; }

/* charts box */
```

```
.chart-box {
  background: linear-gradient(180deg, rgba(255,255,255,0.98), rgba(250,250,255,0.95));
  border-radius: 1rem;
  padding: 1rem;
  box-shadow: 0 8px 30px rgba(2,6,23,0.04);
  height: 280px;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
}

.chart-canvas { max-height: 180px; width: 100%; }

/* savings goal chip */

.goal-chip {
  background: linear-gradient(90deg, rgba(99,102,241,0.08), rgba(139,92,246,0.06));
  border-radius: 0.75rem;
  padding: .6rem .85rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
  border: 1px solid rgba(99,102,241,0.06);
}

/* transactions table */

.table-card {
  background: white;
  border-radius: .8rem;
  overflow: hidden;
  box-shadow: 0 12px 40px rgba(2,6,23,0.04);
  border: 1px solid rgba(15,23,42,0.03);
}

.tx-row: hover { background: #fafafa; }

/* small progress bar */

.progress-sm { height: 10px; background: #eef2ff; border-radius: 999px;
overflow: hidden; }
```

```

    .progress-sm > i { display: block; height: 100%; background: linear-gradient(90deg, var(--primary), var(--primary-2)); width: 0%; transition: width 1.2s ease; }
    /* responsive tweaks */
    @media (max-width: 900px) {
        .chart-box { height: 320px; }
    }
</style>
</head>
<body>
    <!-- NAVBAR -->
    <header class="glass-nav sticky top-0 z-40">
        <div class="max-w-7xl mx-auto px-6 py-4 flex items-center justify-between gap-6">
            <div class="flex items-center gap-6">
                <div class="flex flex-col">
                    <div class="text-2xl font-extrabold text-indigo-700">Hi, <span
id="userName">User</span> 🙋 </div>
                    <div class="text-sm text-gray-500">Welcome back — here's your financial
snapshot</div>
                </div>
            </div>
            <nav class="flex items-center gap-3">
                <button onclick="go('/income')" class="nav-btn"> + Income</button>
                <button onclick="go('/expense')" class="nav-btn"> 💰 Expense</button>
                <button onclick="go('/savingsgoals')" class="nav-btn"> 🎯 Goals</button>
                <button onclick="go('/budget')" class="nav-btn"> 📅 Budget</button>
                <button onclick="go('/financialreport')" class="nav-btn"> 📄 Report</button>
                <button onclick="go('/financial')" class="nav-btn"> 📊 Summary</button>
                <button id="logoutBtn" title="Logout"
                    class="ml-4 px-4 py-2 rounded-lg font-semibold bg-red-500 text-white hover:bg-
red-600 shadow">
                    Logout
                </button>
            </nav>
        </div>
    </body>

```

```

</header>
<!-- MAIN -->
<main class="max-w-7xl mx-auto p-6 grid gap-6">
  <!-- KPI row -->
  <section class="grid grid-cols-1 md:grid-cols-4 gap-6">
    <div class="premium-card">
      <div class="text-sm text-gray-500">Total Income</div>
      <div class="kpi" id="incomeTotal">₹0</div>
      <div class="kpi-sub mt-2 text-gray-500">This month</div>
    </div>
    <div class="premium-card">
      <div class="text-sm text-gray-500">Total Expenses</div>
      <div class="kpi" id="expenseTotal">₹0</div>
      <div class="kpi-sub mt-2 text-gray-500">This month</div>
    </div>
    <div class="premium-card">
      <div class="text-sm text-gray-500">Savings</div>
      <div class="kpi" id="savingsTotal">₹0</div>
      <div class="kpi-sub mt-2 text-gray-500">Available</div>
    </div>
    <div class="premium-card">
      <div class="text-sm text-gray-500">Budget Health</div>
      <div class="kpi" id="budgetHealth">0%</div>
      <div class="kpi-sub mt-2">On track • <span class="text-yellow-
600">✔</span></div>
    </div>
  </section>
  <!-- Charts & lists -->
  <section class="grid grid-cols-1 lg:grid-cols-3 gap-6">
    <!-- Large charts column (spans 2 columns on large screens) -->
    <div class="lg:col-span-2 grid grid-rows-2 gap-6">
      <div class="chart-box">
        <div class="flex items-start justify-between">
          <div>

```

```
<h3 class="text-lg font-semibold">Income vs Expenses</h3>
<p class="text-sm text-gray-500">Monthly overview</p>
</div>
<div class="text-sm text-gray-500">This month</div>
</div>
<canvas id="barChart" class="chart-canvas mt-2"></canvas>
</div>
<div class="chart-box">
  <div class="flex items-start justify-between">
    <div>
      <h3 class="text-lg font-semibold">Expense Breakdown</h3>
      <p class="text-sm text-gray-500">By category</p>
    </div>
    <div class="text-sm text-gray-500">Top categories</div>
  </div>
  <canvas id="donutChart" class="chart-canvas mt-2"></canvas>
</div>
</div>
<!-- Right column: Goals preview + Recent transactions -->
<div class="space-y-6">
  <div class="premium-card">
    <div class="flex items-center justify-between">
      <div>
        <h3 class="text-lg font-semibold"><img alt="Target icon" data-bbox="208 644 228 660"/> Savings Goals</h3>
        <p class="text-sm text-gray-500">Progress toward your goals</p>
      </div>
      <button onclick="go('/savingsgoals')" class="text-sm font-semibold text-indigo-600
hover:underline">View all</button>
    </div>
    <div id="goalsPreview" class="mt-4 grid gap-3">
      <!-- Populated by JS -->
      <div class="text-sm text-gray-500">Loading goals...</div>
    </div>
  </div>
</div>
```

```

<div class="table-card">
  <div class="p-4 flex items-center justify-between border-b">
    <div>
      <h3 class="font-semibold"> 📄 Recent Transactions</h3>
      <div class="text-sm text-gray-500">Latest activity</div>
    </div>
    <button onclick="go('/financial')" class="text-sm px-3 py-1 rounded-md bg-indigo-50 text-indigo-600">See all</button>
  </div>
  <div class="overflow-auto">
    <table class="w-full min-w-[640px]">
      <thead class="bg-white">
        <tr class="text-left text-sm text-gray-600">
          <th class="p-3">Date</th>
          <th class="p-3">Type</th>
          <th class="p-3">Category / Source</th>
          <th class="p-3 text-right">Amount</th>
        </tr>
      </thead>
      <tbody id="transactionsTable" class="text-sm">
        <tr><td class="p-3 text-gray-500">Loading...</td></tr>
      </tbody>
    </table>
  </div>
</div>
</div>
</div>
</section>
</main>
<script>
const API_BASE = "http://localhost:5000/api";
const userId = localStorage.getItem("userId");
const userName = localStorage.getItem("userName") || "User";
// If not logged in, redirect to /login
if (!userId) {

```

```
    window.location.href = "/login";
  }
  document.getElementById("userName").textContent = userName;
  // navigation helpers
  function go(path) {
    if (!path.startsWith("/")) {
      path = "/" + path;
    }
    window.location.href = path;
  }
  document.getElementById("logoutBtn").addEventListener("click", () => {
    localStorage.clear();
    window.location.href = "/login";
  });
  let barChart = null, donutChart = null;
  async function fetchJson(url) {
    const res = await fetch(url);
    if (!res.ok) {
      const txt = await res.text();
      throw new Error(txt || "API error");
    }
    return res.json();
  }
  function fmt(v) {
    if (v === null || v === undefined) return "0";
    const num = Number(v) || 0;
    return num.toLocaleString('en-IN', { maximumFractionDigits: 2 });
  }
  async function loadDashboardData() {
    try {
      const data = await fetchJson(`${API_BASE}/financials/history/${userId}`);
      const totals = data.totals || {};
      const totalIncome = Number(totals.totalIncome || 0);
```



```
const totalExpense = Number(totals.totalExpense || 0);
const totalBudget = Number(totals.totalBudget || 0);
const totalSavings = Number(totals.totalSavings || 0);
const remainingBalance = Number(totals.remainingBalance || (totalIncome -
totalExpense));
document.getElementById("incomeTotal").textContent = "₹" + fmt(totalIncome);
document.getElementById("expenseTotal").textContent = "₹" + fmt(totalExpense);
document.getElementById("savingsTotal").textContent = "₹" + fmt(totalSavings);
// -----
// UPDATED BUDGET HEALTH LOGIC (NO MORE 78%)
// -----
let budgetHealth = 0;
if (totalBudget > 0) {
  // Normal case: budget exists → compare expenses to budget
  budgetHealth = Math.max(
    0,
    Math.min(100, Math.round((1 - (totalExpense / totalBudget)) * 100))
  );
} else {
  // No budget defined
  if (totalIncome === 0 && totalExpense === 0) {
    // No financial data at all → 0%
    budgetHealth = 0;
  } else {
    // Fallback: compute health based on income vs expense
    budgetHealth = totalIncome > 0
      ? Math.max(
        0,
        Math.min(100, Math.round((1 - (totalExpense / totalIncome)) * 100))
      )
      : 0;
  }
}
document.getElementById("budgetHealth").textContent = `${budgetHealth}%`;
```

```
// -----  
// Bar chart: Income vs Expense  
const barCtx = document.getElementById("barChart").getContext("2d");  
if (barChart) barChart.destroy();  
barChart = new Chart(barCtx, {  
  type: "bar",  
  data: {  
    labels: ["Income", "Expenses"],  
    datasets: [{  
      data: [totalIncome, totalExpense],  
      backgroundColor: [  
        getComputedStyle(document.documentElement).getPropertyValue('--primary') ||  
"#6366f1",  
        "#ef4444"  
      ],  
      borderRadius: 10,  
      barThickness: 40  
    }]  
  },  
  options: {  
    responsive: true,  
    maintainAspectRatio: false,  
    plugins: { legend: { display: false } },  
    scales: {  
      y: { beginAtZero: true, ticks: { padding: 6 } },  
      x: { grid: { display: false } }  
    }  
  }  
});  
const categories = {};  
(data.expenses || []).forEach(e => {  
  const cat = e.category || "Other";  
  const amt = Number(e.amount || 0);  
  categories[cat] = (categories[cat] || 0) + Math.abs(amt);
```

```
});
const donutLabels = Object.keys(categories).slice(0, 8);
const donutData = donutLabels.map(l => categories[l]);
const donutCtx = document.getElementById("donutChart").getContext("2d");
if (donutChart) donutChart.destroy();
donutChart = new Chart(donutCtx, {
  type: "doughnut",
  data: {
    labels: donutLabels,
    datasets: [{
      data: donutData,
      backgroundColor: [
        "#6366f1", "#10b981", "#ef4444", "#f59e0b",
        "#3b82f6", "#8b5cf6", "#06b6d4", "#f97316"
      ]
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    cutout: "60%",
    plugins: { legend: { position: 'bottom', labels: { boxWidth: 12, padding: 8 } } }
  }
});

const goalsArea = document.getElementById("goalsPreview");
goalsArea.innerHTML = "";
const goals = data.goals || [];

if (goals.length === 0) {
  goalsArea.innerHTML = '<div class="text-sm text-gray-500">No goals yet — create one from Goals.</div>';
} else {
  goals.slice(0, 6).forEach(g => {
```

```

const saved = Number(g.savedAmount || 0);
const target = Number(g.targetAmount || 0) || 1;
const pct = Math.min(100, Math.round((saved / target) * 100));
const node = document.createElement("div");
node.className = "goal-chip";
node.innerHTML = `
  <div class="flex flex-col">
    <div class="font-semibold text-sm">${escapeHtml(g.goalName || "Goal")}</div>
    <div class="text-xs text-gray-500">₹$ {fmt(saved)} / ₹$ {fmt(target)}</div>
  </div>
  <div style="min-width:120px; text-align:right;">
    <div class="text-sm font-semibold">${pct}%</div>
    <div class="progress-sm mt-2"><i style="width:${pct}%;"></i></div>
  </div>
`;
goalsArea.appendChild(node);
});
}

const txTable = document.getElementById("transactionsTable");
const incomes = (data.incomes || []).map(i => ({ ...i, _type: "Income" }));
const expenses = (data.expenses || []).map(e => ({ ...e, _type: "Expense" }));
const combined = incomes.concat(expenses);
combined.forEach(row => {
  row._dateValue = row.date ? new Date(row.date).getTime() : (row.id || 0);
});
combined.sort((a,b)=> (b._dateValue || 0) - (a._dateValue || 0));
const recent = combined.slice(0, 8);
txTable.innerHTML = "";
if (recent.length === 0) {
  txTable.innerHTML = `<tr><td class="p-3 text-gray-500" colspan="4">No
transactions yet.</td></tr>`;
} else {
  recent.forEach(r => {

```

```

    const amtRaw = Number(r.amount || 0);
    const amtFormatted = (r._type === "Income" ? "+" : "-") + "₹" +
fmt(Math.abs(amtRaw));
    const amtColor = r._type === "Income" ? "text-green-600" : "text-red-600";
    const dateStr = r.date ? (new Date(r.date).toLocaleDateString()) : "-";
    const cat = r._type === "Income"
    ? (r.source || "Salary")
    : (r.category || "Other");
    const tr = document.createElement("tr");
    tr.className = "tx-row";
    tr.innerHTML = `
        <td class="p-3 text-sm text-gray-600">${escapeHtml(dateStr)}</td>
        <td class="p-3 text-sm text-gray-600">${escapeHtml(r._type)}</td>
        <td class="p-3 text-sm text-gray-700">${escapeHtml(String(cat))}</td>
        <td class="p-3 text-sm text-right ${amtColor} font-
semibold">${escapeHtml(amtFormatted)}</td>
    `;
    txTable.appendChild(tr);
  });
}
} catch (err) {
  console.error("Dashboard load error:", err);
  document.getElementById("incomeTotal").textContent = "₹0";
  document.getElementById("expenseTotal").textContent = "₹0";
  document.getElementById("savingsTotal").textContent = "₹0";
  document.getElementById("budgetHealth").textContent = "0%";
  document.getElementById("goalsPreview").innerHTML =
    `<div class="text-sm text-red-500">Failed to load dashboard:
    ${escapeHtml(err.message || "Unknown")}</div>`;
  document.getElementById("transactionsTable").innerHTML =
    `<tr><td class="p-3 text-gray-500" colspan="4">Unable to load
    transactions.</td></tr>`;
}
}

function escapeHtml(unsafe) {

```

```
if (unsafe === null || unsafe === undefined) return "";
return String(unsafe)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;")
    .replaceAll('"', "&#039;");
}
loadDashboardData();
</script>
</body>
</html>
```

Expense.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Add Expense - Budget Tracker</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-gray-100">

  <!-- NAVBAR -->
  <header class="bg-indigo-600 text-white shadow-lg py-4">
    <div class="max-w-7xl mx-auto flex items-center justify-between px-6">
      <h1 class="text-2xl font-bold">Add Expense</h1>

      <nav class="flex gap-3">
        <button onclick="go('/dashboard')"
          class="px-5 py-2 bg-indigo-600 text-white rounded-xl hover:bg-indigo-700
shadow-md">
```

```
        ← Back
    </button>
  </nav>
</div>
</header>

<!-- NAV BTN STYLE -->
<style>
  .nav-btn {
    padding: 8px 14px;
    border-radius: 10px;
    background: rgba(255,255,255,0.2);
    transition: 0.3s;
  }
  .nav-btn:hover { background: rgba(255,255,255,0.35); }
</style>

<!-- MAIN CONTENT -->
<div class="max-w-5xl mx-auto mt-10 p-6">

  <!-- ADD EXPENSE CARD -->
  <div class="bg-white rounded-2xl shadow-xl p-8 border-t-4 border-red-500">
    <h2 class="text-2xl font-bold mb-6 text-gray-800"> 💰 Add New Expense</h2>

    <form id="expenseForm" class="grid grid-cols-1 md:grid-cols-2 gap-6">
      <div>
        <label class="font-semibold text-gray-700">Amount</label>
        <input type="number" id="amount"
          class="mt-2 w-full px-4 py-3 border rounded-xl focus:ring-2 focus:ring-
red-400"
          placeholder="Enter amount" required>
      </div>

      <div>
```

```
<label class="font-semibold text-gray-700">Category</label>
<select id="category"
  class="mt-2 w-full px-4 py-3 border rounded-xl focus:ring-2 focus:ring-
red-400" required>
  <option value="">Select Category</option>
  <option>Food</option>
  <option>Transportation</option>
  <option>Shopping</option>
  <option>Bills</option>
  <option>Entertainment</option>
  <option>Other</option>
</select>
</div>

<div>
  <label class="font-semibold text-gray-700">Description</label>
  <input type="text" id="description"
    class="mt-2 w-full px-4 py-3 border rounded-xl focus:ring-2 focus:ring-
red-400"
    placeholder="Optional">
</div>

<div>
  <label class="font-semibold text-gray-700">Date</label>
  <input type="date" id="date"
    class="mt-2 w-full px-4 py-3 border rounded-xl focus:ring-2 focus:ring-
red-400"
    required>
</div>

<div class="md:col-span-2 mt-4">
  <button type="submit"
    class="w-full bg-red-500 hover:bg-red-600 text-white py-3 rounded-xl
font-semibold shadow-md transition">
    Add Expense
```



```
        </button>
      </div>
    </form>
  </div>

  <!-- EXPENSE HISTORY -->
  <div class="bg-white rounded-2xl shadow-xl p-8 mt-10">
    <h2 class="text-2xl font-bold text-gray-800 mb-6"> 📑 Expense History</h2>

    <table class="w-full border-collapse">
      <thead>
        <tr class="bg-red-500 text-white">
          <th class="p-3 text-left">Category</th>
          <th class="p-3 text-left">Amount</th>
          <th class="p-3 text-left">Description</th>
          <th class="p-3 text-left">Date</th>
          <th class="p-3 text-center">Actions</th>
        </tr>
      </thead>

      <tbody id="expenseTable" class="text-gray-700"></tbody>
    </table>
  </div>

</div>

<!-- SCRIPT -->
<script>
  const API = "http://localhost:5000/api";
  const userId = localStorage.getItem("userId");
  if (!userId) window.location.href = "login.html";

  function go(page) { window.location.href = page; }
```

```
// Load all expenses
async function loadExpenses() {
  const res = await fetch(`${API}/expenses/user/${userId}`);
  const data = await res.json();

  const table = document.getElementById("expenseTable");
  table.innerHTML = "";

  data.forEach(e => {
    const row = `
      <tr class="border-b">
        <td class="p-3">${e.category}</td>
        <td class="p-3 text-red-600 font-semibold">₹${e.amount}</td>
        <td class="p-3">${e.description || "-"}</td>
        <td class="p-3">${e.date}</td>
        <td class="p-3 text-center">
          <button onclick="deleteExpense(${e.id})"
            class="bg-red-500 px-3 py-1 text-white rounded-lg hover:bg-red-
600">
            Delete
          </button>
        </td>
      </tr>
    `;
    table.innerHTML += row;
  });
}

// Add expense
document.getElementById("expenseForm").addEventListener("submit", async (e) =>
{
  e.preventDefault();
```

```
const expense = {
  userId: parseInt(userId),
  amount: parseFloat(document.getElementById("amount").value),
  category: document.getElementById("category").value,
  description: document.getElementById("description").value,
  date: document.getElementById("date").value
};

const res = await fetch(`${API}/expenses`, {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify(expense)
});

if (res.ok) {
  alert("Expense added!");
  document.getElementById("expenseForm").reset();
  loadExpenses();
} else {
  alert("Error adding expense.");
}
});

// Delete expense
async function deleteExpense(id) {
  if (!confirm("Delete this expense?")) return;

  await fetch(`${API}/expenses/${id}`, { method: "DELETE" });
  loadExpenses();
}

loadExpenses();
</script>
```

```
</body>
```

```
</html>
```

Financial.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Financial Dashboard</title>
```

```
  <script src="https://cdn.tailwindcss.com"></script>
```

```
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

```
<style>
```

```
  body {  
    background: #f3f4f6;  
  }
```

```
  .card {  
    background: white;  
    border-radius: 15px;  
    box-shadow: 0 3px 8px rgba(0, 0, 0, 0.08);  
    padding: 20px;  
  }
```

```
  .chart-box {  
    height: 260px;  
    width: 100%;  
  }
```

```
  .section-title {
```

```
        color: #374151;
        font-size: 18px;
        font-weight: 600;
        margin-bottom: 12px;
    }
</style>
</head>
<script>
    function go(page) {
        window.location.href = page;
    }
</script>

<body>

    <!-- NAVBAR -->
    <header class="bg-indigo-600 text-white py-4 shadow">
        <div class="max-w-6xl mx-auto px-4 flex justify-between items-center">
            <h1 class="text-xl font-semibold">Financial Dashboard</h1>
            <nav class="flex gap-3">
                <button onclick="go('/dashboard')"
                    class="px-5 py-2 bg-indigo-600 text-white rounded-xl hover:bg-indigo-700
shadow-md">
                    ← Back
                </button>
            </nav>
        </div>
    </header>

    <div class="max-w-6xl mx-auto py-6 px-4">

        <!-- SUMMARY CARDS -->
        <div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-4 gap-6">
```

```
<div class="card">
  <div class="text-gray-500 text-sm">Total Income</div>
  <div class="text-2xl font-bold text-green-600" id="totalIncome">₹0</div>
</div>
```

```
<div class="card">
  <div class="text-gray-500 text-sm">Total Expense</div>
  <div class="text-2xl font-bold text-red-500" id="totalExpense">₹0</div>
</div>
```

```
<div class="card">
  <div class="text-gray-500 text-sm">Total Savings</div>
  <div class="text-2xl font-bold text-indigo-600" id="totalSavings">₹0</div>
</div>
```

```
<div class="card">
  <div class="text-gray-500 text-sm">Remaining Balance</div>
  <div class="text-2xl font-bold text-yellow-600"
id="remainingBalance">₹0</div>
</div>
```

```
</div>
```

```
<!-- CHARTS -->
```

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-6 mt-8">
```

```
<div class="card">
  <div class="section-title">Income Overview</div>
  <canvas id="incomeChart" class="chart-box"></canvas>
</div>
```

```
<div class="card">
  <div class="section-title">Expense Breakdown</div>
  <canvas id="expenseChart" class="chart-box"></canvas>
```

```
</div>
```

```
</div>
```

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-6 mt-6">
```

```
<div class="card">
```

```
<div class="section-title">Budget Allocation</div>
```

```
<canvas id="budgetChart" class="chart-box"></canvas>
```

```
</div>
```

```
<div class="card">
```

```
<div class="section-title">Savings Goals</div>
```

```
<div id="savingsGoalsList"></div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
const API = "http://localhost:5000/api";
```

```
const userId = localStorage.getItem("userId");
```

```
if (!userId) window.location.href = "login.html";
```

```
// Utility
```

```
const money = (n) => "₹" + Number(n).toLocaleString();
```

```
async function loadDashboard() {
```

```
const res = await fetch(`${API}/financials/history/${userId}`);
```

```
const data = await res.json();
```

```
const totals = data.totals;

// Assign summary card values
document.getElementById("totalIncome").textContent =
money(totals.totalIncome);
document.getElementById("totalExpense").textContent =
money(totals.totalExpense);
document.getElementById("totalSavings").textContent =
money(totals.totalSavings);
document.getElementById("remainingBalance").textContent =
money(totals.remainingBalance);

// INCOME CHART
new Chart(document.getElementById("incomeChart"), {
  type: "bar",
  data: {
    labels: data.incomes.map(i => i.source),
    datasets: [{
      label: "Income",
      data: data.incomes.map(i => i.amount),
      backgroundColor: "#6366F1"
    }]
  },
  options: { plugins: { legend: { display: false } } }
});

// EXPENSE CHART
new Chart(document.getElementById("expenseChart"), {
  type: "pie",
  data: {
    labels: data.expenses.map(e => e.category),
    datasets: [{
      data: data.expenses.map(e => e.amount),
      backgroundColor: ["#EF4444", "#F59E0B", "#10B981", "#3B82F6",
"#8B5CF6"]
    }]
  }
});
```



```
    }]  
  }  
});  
  
// BUDGET DONUT CHART  
new Chart(document.getElementById("budgetChart"), {  
  type: "doughnut",  
  data: {  
    labels: data.budgets.map(b => b.category),  
    datasets: [{  
      data: data.budgets.map(b => b.amount),  
      backgroundColor: ["#4F46E5", "#6366F1", "#818CF8", "#A5B4FC",  
"#C7D2FE"]  
    }]  
  }  
});  
  
// SAVINGS GOALS  
const list = document.getElementById("savingsGoalsList");  
list.innerHTML = "";  
  
data.goals.forEach(g => {  
  const percent = ((g.savedAmount / g.targetAmount) * 100).toFixed(1);  
  
  const div = document.createElement("div");  
  div.className = "mb-4";  
  
  div.innerHTML = `  
    <div class="font-semibold text-gray-700">${g.goalName}</div>  
      <div class="text-sm text-gray-600">₹${g.savedAmount} /  
₹${g.targetAmount}</div>  
    <div class="w-full bg-gray-200 rounded-full h-3 mt-2">  
      <div class="bg-indigo-600 h-3 rounded-full"  
style="width:${percent}%"></div>
```

```
        </div>
    `;

    list.appendChild(div);
  });
}

function logout() {
  localStorage.clear();
  window.location.href = "login.html";
}
loadDashboard();
</script>
</body>
</html>
```

Financialreport.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Financial Report - Budget Tracker</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<script>
  function go(page) {
    window.location.href = page;
  }
</script>

<body class="bg-gray-100">
```

```
<!-- Header Navbar -->
<header class="bg-indigo-600 text-white py-4 shadow-md">
  <div class="max-w-5xl mx-auto px-4 flex justify-between items-center">
    <h1 class="text-2xl font-bold"> 📄 Financial Report</h1>
    <button onclick="go('/dashboard')"
      class="px-5 py-2 bg-indigo-600 text-white rounded-xl hover:bg-indigo-700
shadow-md">
      ← Back
    </button>
  </div>
</header>

<!-- Main Content -->
<div class="max-w-5xl mx-auto px-4 py-10">

  <!-- Download Section -->
  <div
    class="bg-white p-10 rounded-2xl shadow-lg mb-8 border border-gray-200 text-
center hover:shadow-xl transition">
    <h2 class="text-2xl font-bold text-gray-800 mb-3"> 📄 Download Report</h2>
    <p class="text-gray-600 mb-6">Generate a detailed financial report with charts,
expenses, income and more.</p>

    <button onclick="downloadReport()"
      class="bg-indigo-600 hover:bg-indigo-700 text-white px-8 py-3 rounded-xl font-
semibold shadow transition">
      ↓ Download PDF
    </button>
  </div>

  <!-- Email Section -->
  <div
    class="bg-white p-10 rounded-2xl shadow-lg border border-gray-200 text-center
hover:shadow-xl transition">
```

```
<h2 class="text-2xl font-bold text-gray-800 mb-3">✉ Email Report</h2>
  <p class="text-gray-600 mb-4">Send your financial report to any email
address.</p>

  <input id="emailInput" type="email" placeholder="Enter email address"
  class="w-full max-w-sm px-4 py-3 border border-gray-300 rounded-xl focus:ring-
2 focus:ring-indigo-500 mb-4">

  <br>

  <button onclick="sendEmailReport()"
  class="bg-indigo-600 hover:bg-indigo-700 text-white px-8 py-3 rounded-xl font-
semibold shadow transition">
    ✉ Send Report
  </button>
</div>

</div>

<script>
  const userId = localStorage.getItem("userId");
  if (!userId) window.location.href = "login.html";

  function goBack() {
    window.location.href = "dashboard.html";
  }

  function downloadReport() {
    fetch(`http://localhost:5000/api/report/download/${userId}`)
      .then(res => {
        if (!res.ok) throw new Error("Failed to download report");
        return res.blob();
      })
      .then(blob => {
```

```
        const url = window.URL.createObjectURL(blob);
        const a = document.createElement("a");
        a.href = url;
        a.download = "financial_report.pdf";
        a.click();
        window.URL.revokeObjectURL(url);
    })
    .catch(err => alert(err.message));
}

function sendEmailReport() {
    const email = document.getElementById("emailInput").value.trim();
    if (!email) {
        alert("Please enter a valid email");
        return;
    }

    fetch(`http://localhost:5000/api/report/email/${userId}?email=${encodeURIComponent(email)}`, {
        method: 'POST'
    })
    .then(res => res.json())
    .then(data => {
        if (data.error) alert("Error: " + data.error);
        else alert("Email sent successfully!");
    })
    .catch(err => alert("Failed to send email: " + err.message));
}
</script>

</body>

</html>
```

Income.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Add Income - Budget Tracker</title>
  <script src="https://cdn.tailwindcss.com"></script>

  <style>
    .glass-card {
      background: rgba(255, 255, 255, 0.65);
      backdrop-filter: blur(10px);
      border-radius: 20px;
      box-shadow: 0 8px 25px rgba(0, 0, 0, 0.08);
    }

    .table-row:hover {
      background-color: #eef2ff;
      transition: 0.2s;
    }

    .input-box {
      border-radius: 14px;
      padding: 12px;
      background: #f3f4f6;
      border: 1px solid #d1d5db;
      transition: 0.25s;
    }

    .input-box:focus {
      border-color: #6366f1;
```

```
        box-shadow: 0 0 10px rgba(99, 102, 241, 0.3);
        outline: none;
    }
</style>
</head>

<body class="bg-gradient-to-br from-indigo-50 to-blue-50 min-h-screen">

<script>
    function go(page) {
        window.location.href = page;
    }
</script>

<!-- NAVBAR -->

<header class="bg-indigo-600 text-white py-4 shadow-md">
    <div class="max-w-5xl mx-auto px-4 flex justify-between items-center">
        <h1 class="text-2xl font-bold">Add Income</h1>
        <button onclick="go('/dashboard')"
            class="px-5 py-2 bg-indigo-600 text-white rounded-xl hover:bg-indigo-700
shadow-md">
            ← Back
        </button>
    </div>
</header>

</nav>
</div>
</header>

<!-- MAIN BODY -->
<div class="max-w-6xl mx-auto p-6">
```

```

<!-- ADD INCOME FORM -->
<div class="glass-card p-6 mb-10">
    <h2 class="text-xl font-bold text-gray-800 mb-4"> + Add New Income</h2>

    <form id="incomeForm" class="grid grid-cols-1 md:grid-cols-3 gap-5">
        <input id="amount" type="number" placeholder="Amount (₹)" required
class="input-box">
        <input id="source" type="text" placeholder="Income Source" required
class="input-box">
        <input id="date" type="date" class="input-box">

        <button type="submit"
            class="md:col-span-3 py-3 bg-indigo-600 text-white rounded-xl shadow
hover:bg-indigo-700 transition font-semibold">
            Add Income
        </button>
    </form>
</div>

<!-- INCOME LIST -->
<div class="glass-card p-6">
    <h2 class="text-xl font-bold text-gray-800 mb-5"> 📄 Your Income Records</h2>

    <table class="w-full">
        <thead class="bg-indigo-600 text-white rounded-xl">
            <tr>
                <th class="p-3 text-left">Date</th>
                <th class="p-3 text-left">Source</th>
                <th class="p-3 text-left">Amount</th>
                <th class="p-3 text-left">Action</th>
            </tr>
        </thead>
        <tbody id="incomeTable"></tbody>
    </table>

```


</div>

</div>

<script>

```
const API = "http://localhost:5000/api";
```

```
const userId = localStorage.getItem("userId");
```

```
if (!userId) window.location.href = "login.html";
```

```
function goBack() {
```

```
  window.location.href = "dashboard.html";
```

```
}
```

```
// Load Income Records
```

```
async function loadIncome() {
```

```
  const res = await fetch(`${API}/income/user/${userId}`);
```

```
  const data = await res.json();
```

```
  const table = document.getElementById("incomeTable");
```

```
  table.innerHTML = "";
```

```
  data.forEach(item => {
```

```
    const row = document.createElement("tr");
```

```
    row.classList.add("table-row");
```

```
    row.innerHTML = `
```

```
      <td class="p-3">${item.date}</td>
```

```
      <td class="p-3">${item.source}</td>
```

```
      <td class="p-3 font-semibold text-green-600">₹${item.amount}</td>
```

```
      <td class="p-3">
```

```
        <button onclick="deleteIncome(${item.id})"
```

```
          class="px-3 py-1 bg-red-500 text-white rounded-lg hover:bg-red-600">
```

```
                Delete
            </button>
        </td>
    `;

    table.appendChild(row);
});
}

// Add new income
document.getElementById("incomeForm").addEventListener("submit", async (e) =>
{
    e.preventDefault();

    const amount = document.getElementById("amount").value;
    const source = document.getElementById("source").value;
    const date = document.getElementById("date").value;

    const payload = {
        user_id: parseInt(userId),
        amount: parseFloat(amount),
        source,
        date
    };

    await fetch(`${API}/income`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload)
    });

    document.getElementById("incomeForm").reset();
    loadIncome();
});
```

```
// Delete income
async function deleteIncome(id) {
  if (!confirm("Delete this income entry?")) return;

  await fetch(`${API}/income/${id}`, { method: "DELETE" });
  loadIncome();
}

loadIncome();
</script>

</body>

</html>
```

Index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width,initial-scale=1" />
<title>Budget Tracker - Auth</title>
<style>
      body{font-family:Arial,Helvetica,sans-serif;background:linear-
gradient(135deg,#667eea,#764ba2);height:100vh;margin:0;display:flex;align-
items:center;justify-content:center}
      .card{width:360px;background:#fff;padding:28px;border-radius:12px;box-shadow:0
10px 30px rgba(0,0,0,0.2)}
      h2{text-align:center;margin:0 0 18px}
      input{width:100%;padding:10px;margin:8px 0;border-radius:8px;border:1px solid
#cfecfc}
      button{width:100%;padding:12px;margin-top:10px;border-
radius:8px;border:0;background:#3b49d8;color:#fff;font-size:15px;cursor:pointer}
```

```
.small{font-size:13px;text-align:center;margin-top:10px}
.hidden{display:none}
</style>
</head>
<body>
  <div class="card">
    <!-- Register form -->
    <div id="registerBox">
      <h2>Create Account</h2>
      <form id="registerForm">
        <input id="regName" placeholder="Full name" required />
        <input id="regEmail" type="email" placeholder="Email" required />
        <input id="regPassword" type="password" placeholder="Password" required />
        <button type="submit">Register</button>
      </form>
      <div class="small">Already have an account? <a id="showLogin"
href="#">Login</a></div>
    </div>

    <!-- Login form -->
    <div id="loginBox" class="hidden">
      <h2>Login</h2>
      <form id="loginForm">
        <input id="loginEmail" type="email" placeholder="Email" required />
        <input id="loginPassword" type="password" placeholder="Password" required />
        <button type="submit">Login</button>
      </form>
      <div class="small">Don't have account? <a id="showRegister"
href="#">Register</a></div>
    </div>

    <!-- Verify form -->
    <div id="verifyBox" class="hidden">
      <h2>Verify Email</h2>
```

```
<form id="verifyForm">
  <input id="verifyEmail" type="email" placeholder="Email" readonly />
  <input id="verifyCode" placeholder="Verification code" required />
  <button type="submit">Verify</button>
</form>

<div class="small"><a id="backToLogin" href="#">Back to Login</a></div>
</div>
</div>

<script>
const API = "

function el(id){ return document.getElementById(id); }

function show(name){
  el("registerBox").classList.add("hidden");
  el("loginBox").classList.add("hidden");
  el("verifyBox").classList.add("hidden");
  if(name==="register") el("registerBox").classList.remove("hidden");
  if(name==="login") el("loginBox").classList.remove("hidden");
  if(name==="verify") el("verifyBox").classList.remove("hidden");
}

async function parseResponse(res){
  const text = await res.text();
  try { return JSON.parse(text); } catch { return { raw: text }; }
}

/* REGISTER */
el("registerForm").addEventListener("submit", async(e)=>{
  e.preventDefault();
  const payload={
    name: el("regName").value,
```

```
    email: el("regEmail").value.toLowerCase(),
    password: el("regPassword").value
  };

  const res=await fetch("/api/register",{method:"POST",headers: {"Content-Type":"application/json"},body:JSON.stringify(payload)});
  const body=await parseResponse(res);
  if(!res.ok) return alert(body.error||"Register failed");
  alert("Registered! Enter OTP");
  el("verifyEmail").value=payload.email;
  show("verify");
});

/* VERIFY */
el("verifyForm").addEventListener("submit", async(e)=>{
  e.preventDefault();
  const payload={
    email: el("verifyEmail").value,
    code: el("verifyCode").value
  };

  const res=await fetch("/api/verify",{method:"POST",headers: {"Content-Type":"application/json"},body:JSON.stringify(payload)});
  const body=await parseResponse(res);
  if(!res.ok) return alert(body.error||"Verify failed");
  alert("Verified! Login now.");
  show("login");
});

/* LOGIN — FIXED */
el("loginForm").addEventListener("submit", async(e)=>{
  e.preventDefault();
  const payload={
    email: el("loginEmail").value.toLowerCase(),
    password: el("loginPassword").value
  };
});
```

```
const res=await fetch("/api/login",{
  method:"POST",
  headers:{"Content-Type":"application/json"},
  body:JSON.stringify(payload)
});

const body=await parseResponse(res);
if(!res.ok) return alert(body.error||"Login failed");

/* FIX #1 — backend returns {id,name,email}, not body.user */
localStorage.setItem("userId", body.id);
localStorage.setItem("userName", body.name);
localStorage.setItem("userEmail", body.email);

alert("Welcome " + body.name);

/* FIX #2 — correct redirect route */
window.location.href = "/dashboard";
});

show("register");
</script>
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Login - Budget Tracker</title>

<!-- TailwindCSS -->
<script src="https://cdn.tailwindcss.com"></script>

<style>
  /* Fade animation */
  .fade-in {
    animation: fadeIn 0.4s ease-in-out;
  }

  @keyframes fadeIn {
    from { opacity: 0; transform: translateY(10px); }
    to { opacity: 1; transform: translateY(0); }
  }
</style>
</head>

<body class="bg-gradient-to-br from-indigo-500 via-purple-500 to-pink-500 min-h-screen
flex justify-center items-center">

  <!-- Main Card -->
  <div class="backdrop-blur-xl bg-white/20 shadow-2xl rounded-3xl p-10 w-full max-w-
md fade-in border border-white/30">

    <h1 class="text-4xl font-extrabold text-white text-center mb-3 drop-shadow-lg">
      Budget Tracker
    </h1>

    <p class="text-center text-white/80 mb-8">Manage your money smartly</p>

    <h2 class="text-xl text-white font-semibold mb-4 text-center">Login</h2>
```



```
<div class="space-y-4">
  <input id="email" type="email" placeholder="Email"
    class="w-full px-4 py-3 rounded-xl bg-white/20 border border-white/40 text-
white placeholder-white/60 focus:ring-2 focus:ring-white" />

  <input id="password" type="password" placeholder="Password"
    class="w-full px-4 py-3 rounded-xl bg-white/20 border border-white/40 text-
white placeholder-white/60 focus:ring-2 focus:ring-white" />

  <button onclick="login()"
    class="w-full bg-white/20 backdrop-blur-xl border border-white/40 hover:bg-
white/30 text-white py-3 rounded-xl font-semibold shadow-lg transition">
    Login
  </button>
</div>

<p class="mt-6 text-center text-white/80">
  Don't have an account?
  <button onclick="openRegister()" class="text-white font-semibold underline
hover:text-gray-200">
    Register
  </button>
</p>
</div>

<!-- REGISTER MODAL -->
<div id="registerModal" class="fixed inset-0 hidden bg-black bg-opacity-60 flex items-
center justify-center">
  <div class="bg-white rounded-2xl p-8 w-full max-w-md shadow-xl fade-in">
    <h2 class="text-2xl font-bold text-indigo-600 mb-4 text-center">Create
Account</h2>

    <input id="regName" type="text" placeholder="Full Name"
      class="w-full px-4 py-3 mb-3 bg-gray-100 rounded-xl border border-gray-300
focus:ring-2 focus:ring-indigo-500" />
```

```
<input id="regEmail" type="email" placeholder="Email"
      class="w-full px-4 py-3 mb-3 bg-gray-100 rounded-xl border border-gray-300
focus:ring-2 focus:ring-indigo-500" />
```

```
<input id="regPassword" type="password" placeholder="Password"
      class="w-full px-4 py-3 mb-3 bg-gray-100 rounded-xl border border-gray-300
focus:ring-2 focus:ring-indigo-500" />
```

```
<button onclick="registerUser()"
      class="w-full bg-indigo-600 text-white py-3 rounded-xl hover:bg-indigo-700
font-semibold shadow-md">
```

Send OTP

```
</button>
```

```
<button onclick="closeRegister()"
```

```
      class="mt-3 w-full bg-gray-300 py-2 rounded-xl hover:bg-gray-400">
```

Cancel

```
</button>
```

```
</div>
```

```
</div>
```

```
<!-- OTP MODEL -->
```

```
<div id="otpModal" class="fixed inset-0 hidden bg-black bg-opacity-60 flex items-
center justify-center">
```

```
<div class="bg-white rounded-2xl p-8 w-full max-w-md shadow-xl fade-in">
```

```
<h2 class="text-2xl font-bold text-indigo-600 mb-4 text-center">Verify OTP</h2>
```

```
<input id="otpInput" type="number" placeholder="Enter OTP"
```

```
      class="w-full px-4 py-3 mb-4 bg-gray-100 rounded-xl border border-gray-300
focus:ring-2 focus:ring-indigo-500" />
```

```
<button onclick="verifyOtp()"
```

```
      class="w-full bg-indigo-600 hover:bg-indigo-700 text-white py-3 rounded-xl
font-semibold shadow-md">
```

```
        Verify
    </button>
</div>
</div>

<!-- SCRIPT -->
<script>
    const API = "http://localhost:5000/api";

    function openRegister() {
        document.getElementById("registerModal").classList.remove("hidden");
    }

    function closeRegister() {
        document.getElementById("registerModal").classList.add("hidden");
    }

    function registerUser() {
        const name = document.getElementById("regName").value;
        const email = document.getElementById("regEmail").value;
        const password = document.getElementById("regPassword").value;

        fetch(`${API}/register`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ name, email, password })
        })
        .then(res => res.json())
        .then(data => {
            if (data.error) return alert(data.error);

            closeRegister();
            document.getElementById("otpModal").classList.remove("hidden");
        })
    }
}
```

```
    });  
  }  
  
  function verifyOtp() {  
    const email = document.getElementById("regEmail").value;  
    const code = document.getElementById("otpInput").value;  
  
    fetch(`${API}/verify`, {  
      method: "POST",  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify({ email, code })  
    })  
      .then(res => res.json())  
      .then(data => {  
        if (data.error) return alert(data.error);  
  
        alert("Verification successful! Please login.");  
        document.getElementById("otpModal").classList.add("hidden");  
      });  
  }  
  
  function login() {  
    const email = document.getElementById("email").value;  
    const password = document.getElementById("password").value;  
  
    fetch(`${API}/login`, {  
      method: "POST",  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify({ email, password })  
    })  
      .then(res => res.json())  
      .then(data => {  
        if (data.error) return alert(data.error);  
      });  
  }  
}
```

```
        localStorage.setItem("userId", data.user.id);
        localStorage.setItem("userName", data.user.name);

        window.location.href = "/dashboard";
    });
}
</script>

</body>
</html>
```

Savingsgoals.pdf

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Savings Goals - Budget Tracker</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>

<script>
  function go(page) {
    window.location.href = page;
  }
</script>

<body class="bg-gray-100">

  <!-- NAVBAR -->
```

```

<header class="bg-indigo-600 text-white py-4 shadow-md">
  <div class="max-w-5xl mx-auto px-4 flex justify-between items-center">
    <h1 class="text-2xl font-bold">Savegoals</h1>
    <button onclick="go('/dashboard')"
      class="px-5 py-2 bg-indigo-600 text-white rounded-xl hover:bg-indigo-700
shadow-md">
      ← Back
    </button>
  </div>
</header>
<!-- MAIN CONTENT -->
<div class="max-w-6xl mx-auto px-4 py-8">

  <!-- ADD GOAL FORM -->
  <div class="bg-white p-8 rounded-2xl shadow-lg mb-10">
    <h2 class="text-xl font-bold text-gray-800 mb-4">+ Add New Savings Goal</h2>

    <form id="goalForm" class="grid grid-cols-1 md:grid-cols-2 gap-6">
      <div>
        <label class="block text-gray-600 mb-1 font-medium">Goal Name</label>
        <input id="goalName" type="text"
          class="w-full px-4 py-3 rounded-xl border border-gray-300 focus:ring-2
focus:ring-indigo-500"
          placeholder="e.g., New Phone" required>
      </div>

      <div>
        <label class="block text-gray-600 mb-1 font-medium">Target Amount</label>
        <input id="targetAmount" type="number"
          class="w-full px-4 py-3 rounded-xl border border-gray-300 focus:ring-2
focus:ring-indigo-500"
          placeholder="e.g., 50000" required>
      </div>
    </form>
  </div>

```

```
<div>
  <label class="block text-gray-600 mb-1 font-medium">Saved Amount</label>
  <input id="savedAmount" type="number"
    class="w-full px-4 py-3 rounded-xl border border-gray-300 focus:ring-2
focus:ring-indigo-500"
    placeholder="e.g., 12000" required>
</div>

<div>
  <label class="block text-gray-600 mb-1 font-medium">Deadline</label>
  <input id="deadline" type="date"
    class="w-full px-4 py-3 rounded-xl border border-gray-300 focus:ring-2
focus:ring-indigo-500"
    required>
</div>

<button type="submit"
  class="md:col-span-2 mt-3 bg-indigo-600 hover:bg-indigo-700 text-white py-
3 rounded-xl font-semibold shadow transition">
  Add Goal
</button>
</form>
</div>

<!-- GOALS LIST -->
<div class="bg-white p-8 rounded-2xl shadow-lg">
  <h2 class="text-xl font-bold text-gray-800 mb-4"> 📋 Your Savings Goals</h2>

  <div id="goalsContainer" class="space-y-4">
    <!-- Cards generated dynamically -->
  </div>
</div>
</div>
```

```
<script>
  const API_BASE_URL = "http://localhost:5000/api/savingsgoals";

  const userId = localStorage.getItem("userId");
  const userName = localStorage.getItem("userName");

  if (!userId) window.location.href = "login.html";

  // Load goals
  async function loadGoals() {
    const res = await fetch(`${API_BASE_URL}/user/${userId}`);
    const data = await res.json();

    const container = document.getElementById("goalsContainer");
    container.innerHTML = "";

    data.forEach(g => {
      const progress = ((g.savedAmount / g.targetAmount) * 100).toFixed(2);

      const card = document.createElement("div");
      card.className =
        "p-6 bg-gray-50 rounded-xl border shadow flex flex-col md:flex-row justify-between gap-4";

      card.innerHTML = `
        <div class="flex-1">
          <h3 class="text-lg font-bold text-indigo-600">${g.goalName}</h3>
          <p class="text-gray-600 mt-1">Target: ₹${g.targetAmount}</p>
          <p class="text-gray-600">Saved: ₹${g.savedAmount}</p>
          <p class="text-gray-600">Deadline: ${g.deadline}</p>

          <div class="w-full bg-gray-200 rounded-full h-3 mt-4">
            <div class="bg-indigo-600 h-3 rounded-full transition-all" style="width:
      ${progress}%;"></div>
        </div>
      `;

      container.appendChild(card);
    });
  }

  loadGoals();
</script>
```



```
        </div>
        <p class="text-gray-700 text-sm mt-1 font-semibold">${progress}%
completed</p>
    </div>

    <button class="bg-red-500 hover:bg-red-600 text-white px-4 py-2 rounded-xl
shadow self-start"
        onclick="deleteGoal(${g.id})">
        Delete
    </button>
    `;

    container.appendChild(card);
    });
}

// Add goal
document.getElementById("goalForm").addEventListener("submit", async e => {
    e.preventDefault();

    const goalData = {
        user_id: parseInt(userId),
        goalName: goalName.value,
        targetAmount: parseFloat(targetAmount.value),
        savedAmount: parseFloat(savedAmount.value),
        deadline: deadline.value
    };

    await fetch(API_BASE_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(goalData)
    });
});
```

```
        goalForm.reset();
        loadGoals();
    });

    // Delete goal
    async function deleteGoal(id) {
        if (!confirm("Delete this goal?")) return;

        await fetch(`${API_BASE_URL}/${id}`, { method: "DELETE" });
        loadGoals();
    }

    function goBack() {
        window.location.href = "dashboard.html";
    }

    loadGoals();
</script>

</body>

</html>
```

.env file

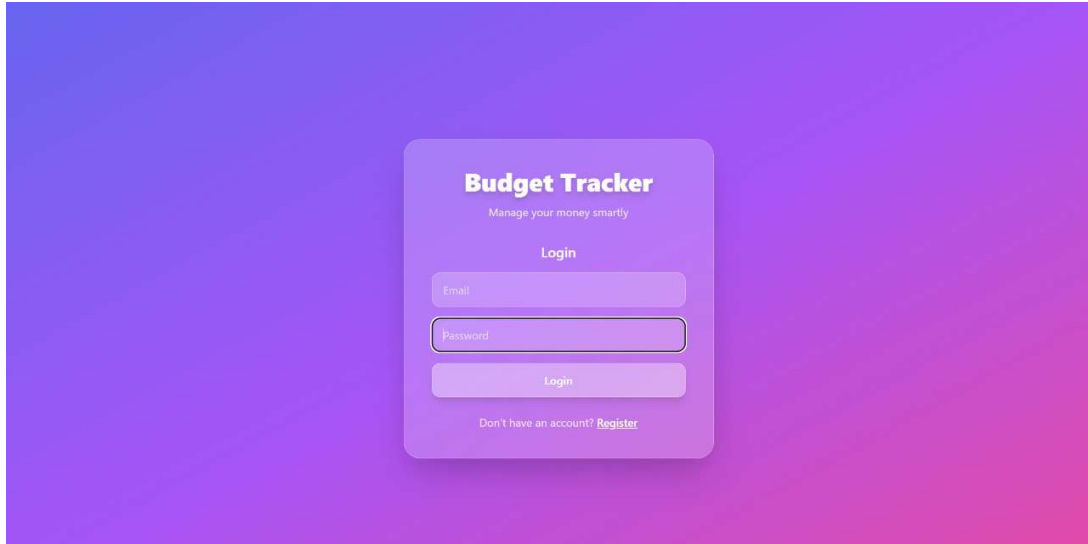
```
FLASK_SECRET=some_long_random_string
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=financialanalyzerbot@gmail.com
SMTP_PASS=cvay xcfh xsha hmmy
SMTP_USE_TLS=1
```

CHAPTER 7

RESULTS

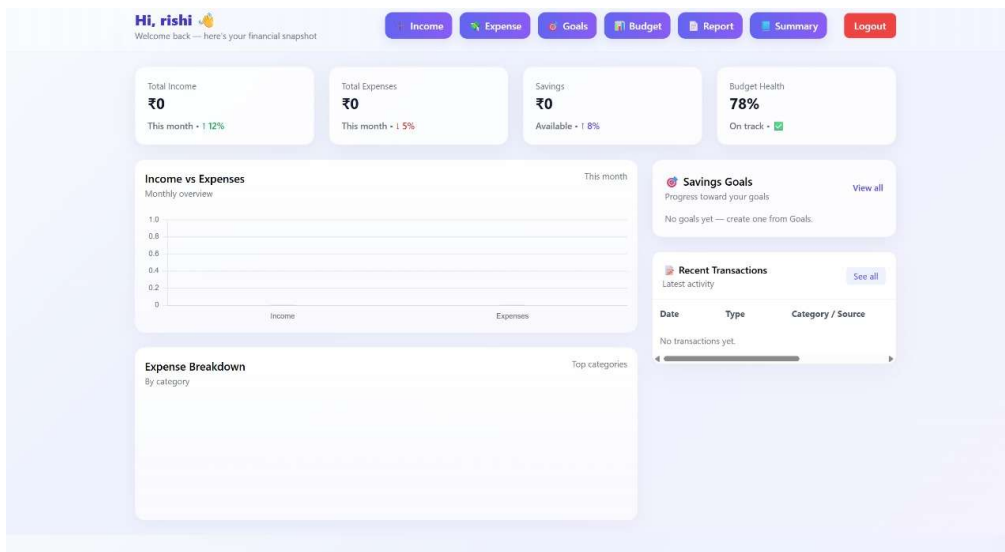
7.1 Login Page

The login screen is clean and easy to understand. Users can enter their email and password to access their account. The page also includes a simple link to register if the user does not have an account. Everything loads smoothly, and the interface feels modern and welcoming.



7.2 Dashboard

After logging in, the user is taken to the dashboard. This page gives a quick summary of their money situation, such as income, expenses, savings, and budget health. Graphs and small cards help users understand their spending at a glance.



7.3 Add Income Page

This section allows users to enter the money they earn. The form asks for the amount, the source of income, and the date. Once the user submits, the new income entry appears in the table below.

Add Income
Back

+ Add New Income

Amount (₹)
Income Source
dd-mm-yyyy

Add Income

Your Income Records

Date	Source	Amount	Action
------	--------	--------	--------

7.4 Add Expense Page

Users can add their daily expenses on this page. They enter the amount, category, description, and date. The added expense immediately appears in the expense history table.

Add Expense
Back

+ Add New Expense

Amount
Category

Enter amount
Select Category

Description
Date

Optional
dd-mm-yyyy

Add Expense

Expense History

Category	Amount	Description	Date	Actions
----------	--------	-------------	------	---------

7.5 Savings Goals Page

This page helps users set and track personal savings goals. They can add a goal name, target amount, saved amount, and deadline. All goals appear below the form, making progress easy to follow.

The screenshot shows the 'Savegoals' page with a blue header bar containing the title 'Savegoals' and a '← Back' button. Below the header is a white card titled '+ Add New Savings Goal'. This card contains four input fields: 'Goal Name' (placeholder: e.g., New Phone), 'Target Amount' (placeholder: e.g., 50000), 'Saved Amount' (placeholder: e.g., 12000), and 'Deadline' (placeholder: dd-mm-yyyy with a calendar icon). A blue 'Add Goal' button is positioned below these fields. Underneath the form card is another white card titled 'Your Savings Goals' with a small icon.

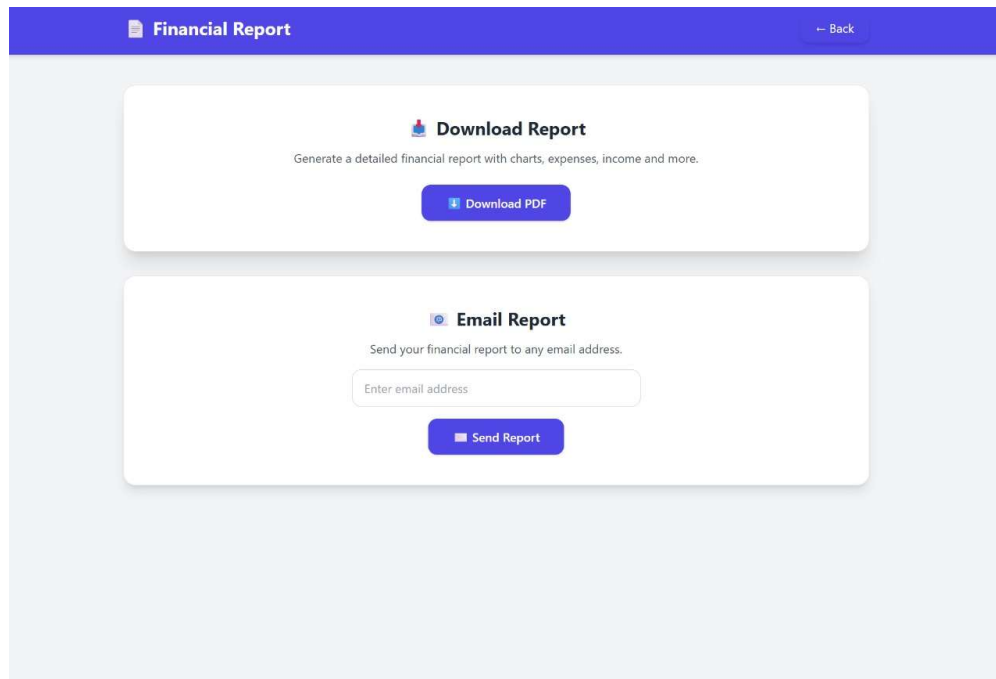
7.6 Budget Page

The budget page allows users to set a spending limit for different categories. After entering the budget amount, it is displayed in a list. These values are later used to calculate budget health on the dashboard.

The screenshot shows the 'Budgets' page with a blue header bar containing the title 'Budgets' and a '← Back' button. Below the header is a white card titled '+ Add New Budget'. This card contains two input fields: 'Category' (placeholder: e.g., Food, Travel) and 'Amount' (placeholder: e.g., 20000). A blue 'Add Budget' button is positioned below these fields. Underneath the form card is another white card titled 'Your Budgets' with a small icon.

7.7 Financial Report Page

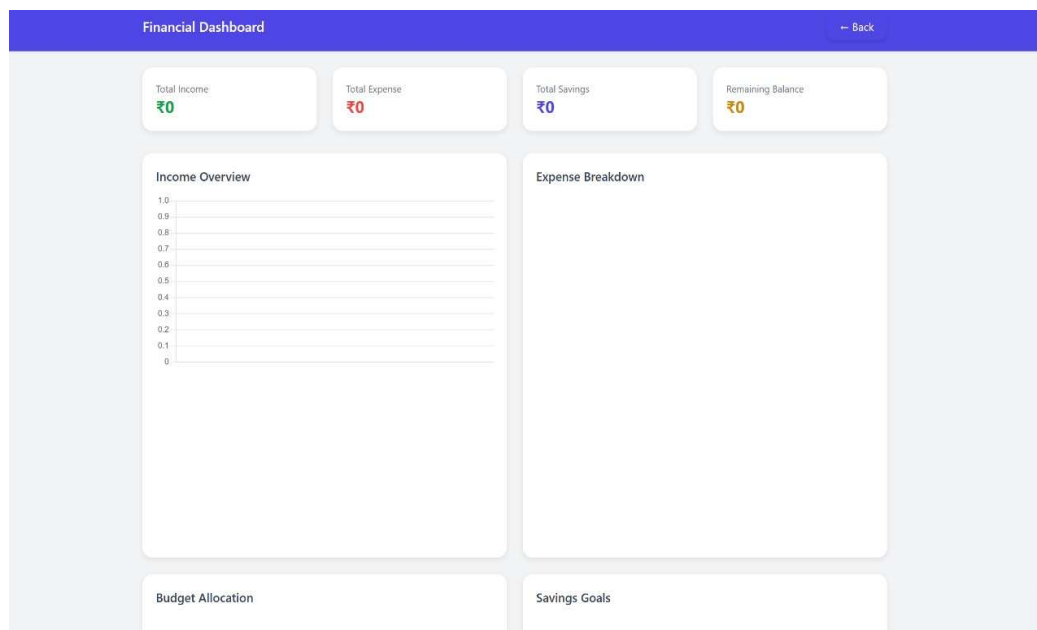
This section lets users generate a detailed financial report. They can either download the report as a PDF or send it to an email address. Both features work correctly, showing that PDF generation and email sending are successfully implemented.



The screenshot shows the 'Financial Report' page with a blue header bar containing a document icon, the title 'Financial Report', and a 'Back' link. The main content area has a light blue background and contains two white rounded rectangular boxes. The first box is titled 'Download Report' with a printer icon and a subtext 'Generate a detailed financial report with charts, expenses, income and more.' Below this is a blue button with a PDF icon and the text 'Download PDF'. The second box is titled 'Email Report' with an envelope icon and a subtext 'Send your financial report to any email address.' Below this is a text input field with the placeholder 'Enter email address' and a blue button with an envelope icon and the text 'Send Report'.

7.8 Financial Summary Page

This page provides a complete view of the user's finances. It shows total income, expenses, savings, and the remaining balance. Graphs help users understand their financial trends more clearly.



CHAPTER 8

CONCLUSION

The Smart Financial Planning and Budget Tracking System shows how technology can make managing money simple and effective. By combining secure login, automated tracking, and real-time analytics, it removes the hassle of manual calculations and helps users stay in control of their finances.

The system achieves its main goals: it records and organizes income and expenses, gives clear spending insights through charts and summaries, monitors budgets automatically, tracks savings goals with progress bars, and generates professional PDF reports that can be downloaded or shared by email.

From a technical perspective, the project highlights the strength of a Flask + SQLite backend working with a responsive frontend. Secure APIs, smooth routing, and a well-structured database design ensure safe data handling and reliable performance.

Overall, the Smart Financial Planning System is more than just a tracker—it acts like a financial assistant. It turns raw transactions into useful insights, helping students, families, and professionals take charge of their money and plan smarter for the future. It also shows how lightweight tools can deliver powerful results without heavy infrastructure, making the system practical for everyday use and adaptable for future improvements.

REFERENCES

8.1 Academic & Algorithmic References

1. Miller, G. A. (1956). *The Magical Number Seven, Plus or Minus Two*. *Psychological Review*, 63(2), 81–97. (A classic study on how people process information, guiding the design of simple, user-friendly interfaces.)
2. Atkinson, R. C., & Shiffrin, R. M. (1968). *Human Memory: A Proposed System and Its Control Processes*. In K.W. Spence (Ed.), *The Psychology of Learning and Motivation*. (Used to understand how users recall and manage financial data.)
3. Saltzer, J. H., & Schroeder, M. D. (1975). *The Protection of Information in Computer Systems*. *Communications of the ACM*. (Provided the foundation for authentication and security principles applied in OTP verification.)
4. RFC 5321 – *Simple Mail Transfer Protocol (SMTP)*. Internet Engineering Task Force (IETF). (Defines the protocol used for secure OTP email delivery.)

8.2 Technical Documentation

1. Flask Documentation – Pallets Project. Available at <https://flask.palletsprojects.com/>
2. SQLite Documentation – SQLite Consortium. Available at <https://sqlite.org/docs.html>
3. TailwindCSS Documentation – Tailwind Labs. Available at <https://tailwindcss.com/docs>
4. Chart.js Documentation – Chart.js Contributors. Available at <https://www.chartjs.org/docs/latest/>
5. ReportLab PDF Toolkit – ReportLab Developers. Available at <https://www.reportlab.com/documentation/>
6. Python smtplib & email.message Libraries – Python Software Foundation. Available at <https://docs.python.org/3/library/smtplib.html> and <https://docs.python.org/3/library/email.message.html>

8.3 Libraries & Tools

- **Flask Framework**–Handles backend routing, authentication, and API development.
- **SQLite3** – Lightweight SQL database for storing income, expenses, budgets, and savings goals.
- **TailwindCSS** – Utility-first CSS framework for clean and modern UI styling.
- **Chart.js** – Creates interactive bar, pie, and donut charts for dashboards.
- **ReportLab** – Generates professional PDF financial reports.
- **SMTP (Gmail)** – Sends OTPs and email-based PDF reports.
- **JavaScript Fetch API** – Enables smooth communication between frontend and backend.
- **HTML5 / CSS3 / JavaScript** – Core technologies for structure, styling, and interactivity.
- **VS Code** – Used as the main development environment.